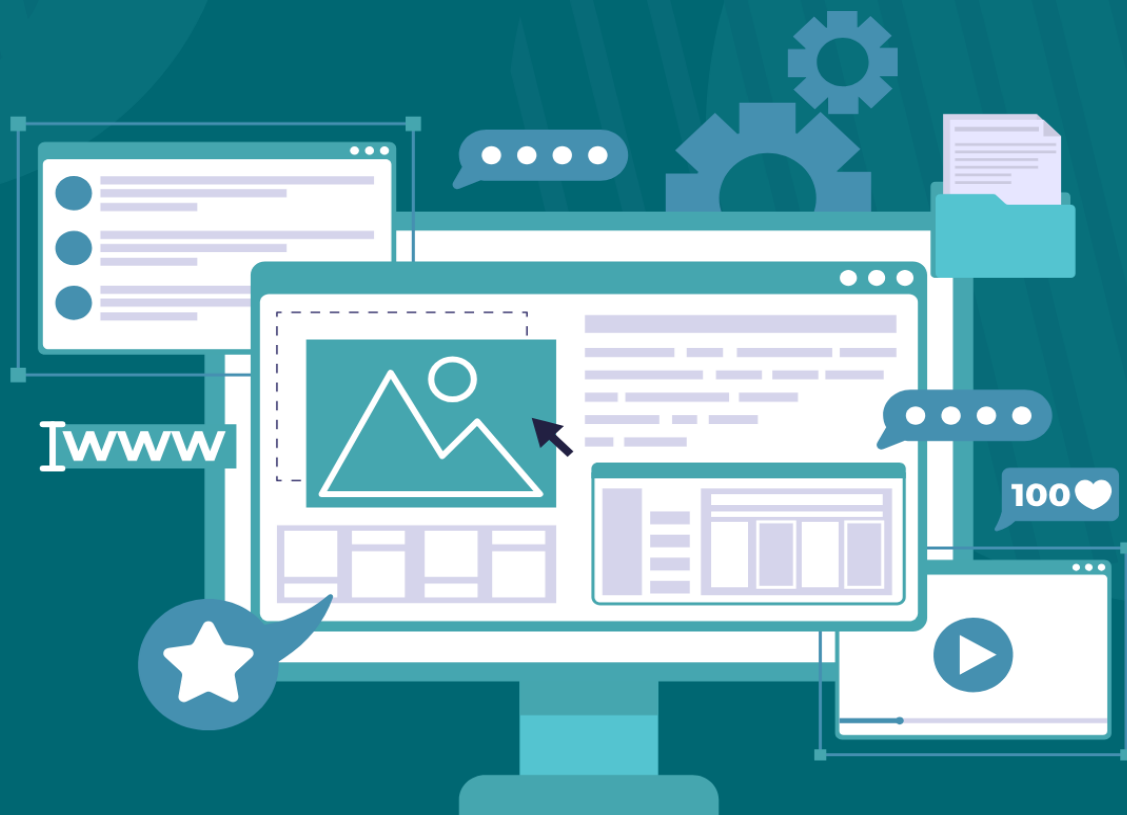


GCWUS



WEB DESIGN & DEVELOPMENT

LAB MANUAL 14

Submitted By :

Manahil Abid 10
Eman Nisar Ahmad 43

Submitted To :

Ma'am Shaista
Computer Science

27-02-2026

Form Creation

Task:

Objective:
Develop a simple ASP.NET Core MVC application using Entity Framework Core (Code First approach) to store Student/User registration data into SQL Server.

The application must allow a user to submit a form containing:

- Name
- Email
- Password
- Gender
- Age

After successful submission, data should be saved in the database and a success alert message should be shown.

Step 1: Install Required Packages

- Create a new ASP.NET Core MVC project in Visual Studio.
- Install NuGet packages:
 - Microsoft.EntityFrameworkCore
 - Microsoft.EntityFrameworkCore.SqlServer
 - Microsoft.EntityFrameworkCore.Tools

Step 2: Create Model Class

- Create a Models folder.
- Create class Student with properties: Id, Name, Email, Password, Gender, Age.
- Apply data annotations (Required, etc.)

Step 3: Create DbContext Class

- Create class StudentDbContext inheriting from DbContext.
- Accept DbContextOptions in constructor.
- Add DbSet<Student> Students { get; set; }

Step 4: Add Connection String in appsettings.json

- Add connection string with key DefaultConnection
- "ConnectionStrings": {
 "DefaultConnection": "Server=your_server_name;Database=your_database_name;Trusted_Connection=True;TrustServerCertificate=True;"
}

Step 5: Register Connection String in Program.cs

- Read connection string from configuration.
- Register DbContext using AddDbContext and SQL Server provider.
- var connectionString = builder.Configuration.GetConnectionString("DefaultConnection");
builder.Services.AddDbContext<ApplicationDbContext>(options =>
 options.UseSqlServer(connectionString));

Step 6: Migration & Update Database

- Use Package Manager Console or CLI:
 - Add-Migration InitialCreate
 - Update-Database

Step 7: Controller Injection and Create User Form Page (Create Template)

- Controller with constructor injection of DbContext.
- Scaffold Create Razor page for Student.
- Fields: Name, Email, Password, Gender, Age.

Step 8: HTTP POST Action to Submit Data

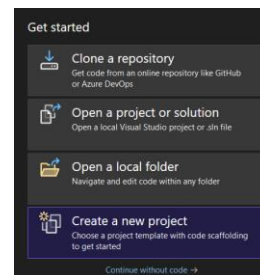
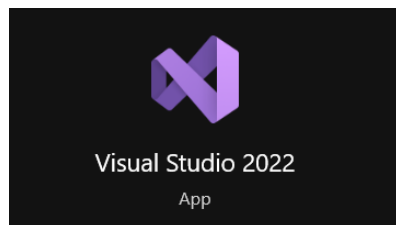
- [HttpPost] action accepting Student model.
- Validate ModelState.
- Save data using DbContext.

Step 9: Show Success Alert Message

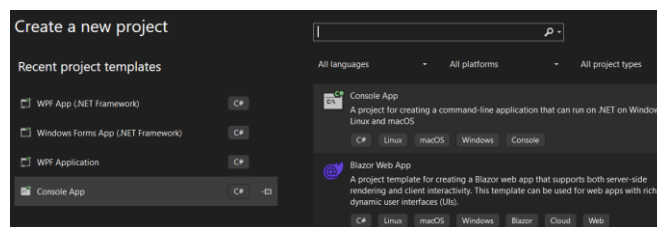
- After successful submission, show alert: "Data successfully submitted" using TempData or JavaScript.

Step 10: Design the User Form

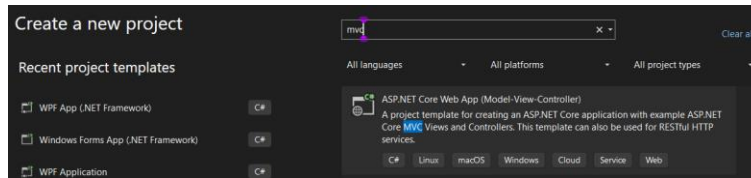
- Search & open **Visual Studio 2022**.



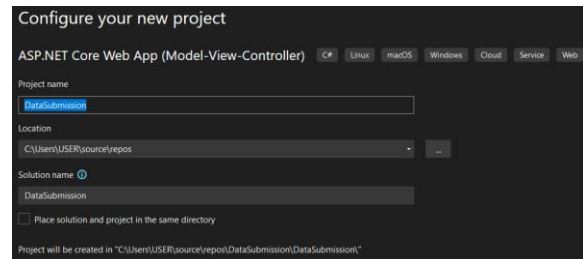
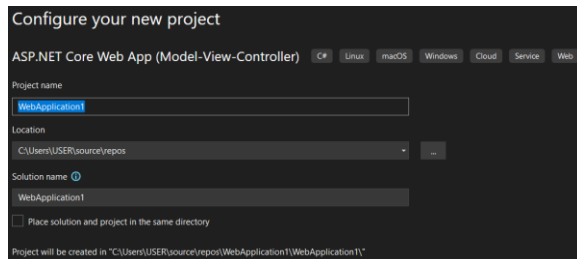
- Click on **Create a new Project**:



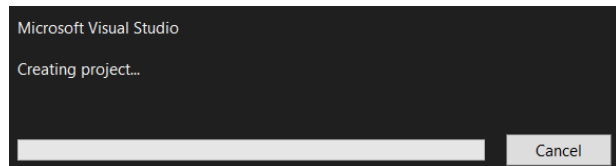
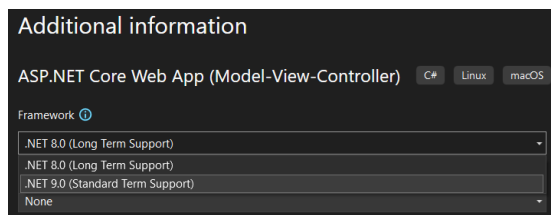
- Search **MVC** in the search bar:



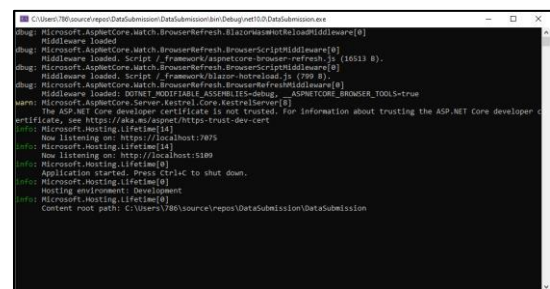
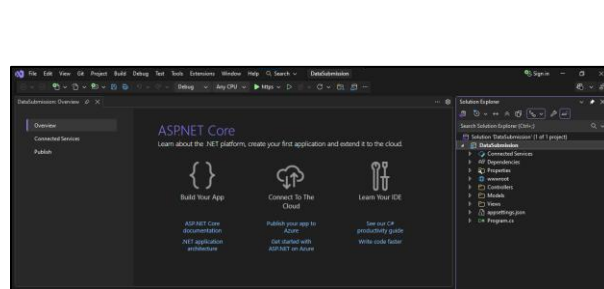
- Choose name for your project Like **DataSubmission**:



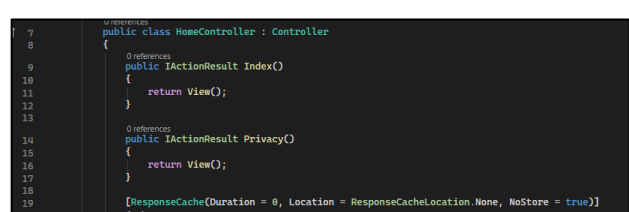
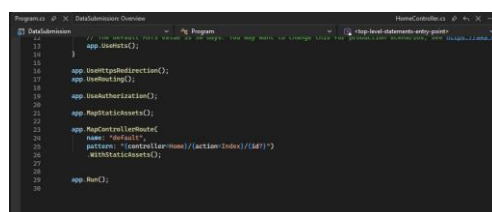
- Choose **version** & next:



- ASP.Net Core **interface**:



- Firstly **program.cs** file run then in it **the Home controller index method** will run and it return the index view which is actually the html page in home tab. The **Action method** runs.



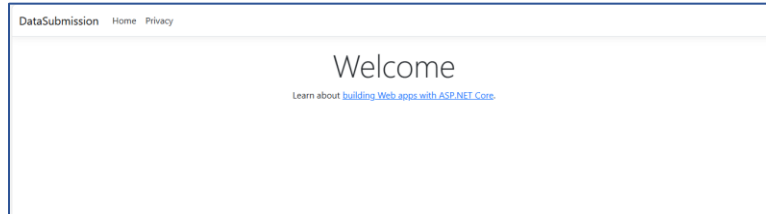
- **Model file** will be html file which shows actually

```

Program.cs      DataSubmission: Overview      Index.cshtml
1      @{
2          ViewData["Title"] = "Home Page";
3      }
4
5      <div class="text-center">
6          <h1 class="display-4">Welcome</h1>
7          <p>Learn about <a href="https://learn.microsoft.com/aspnet/core">building Web apps with ASP.NET Core</a>.</p>
8      </div>
9

```

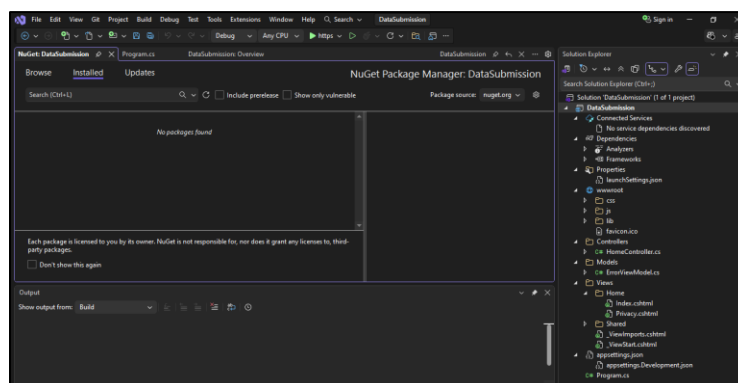
- **Run** the code this screen will appear.



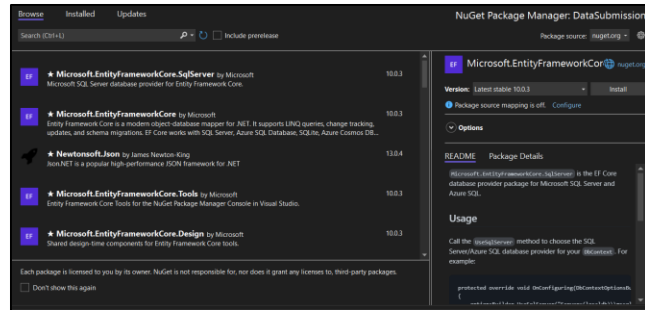
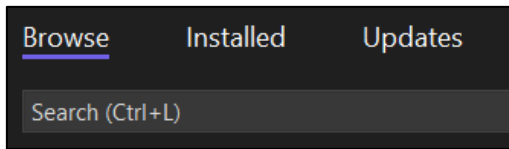
Step 1: Install Required Packages

- Create a new ASP.NET Core MVC project in Visual Studio.
- Install NuGet packages:
 - Microsoft.EntityFrameworkCore
 - Microsoft.EntityFrameworkCore.SqlServer
 - Microsoft.EntityFrameworkCore.Tools

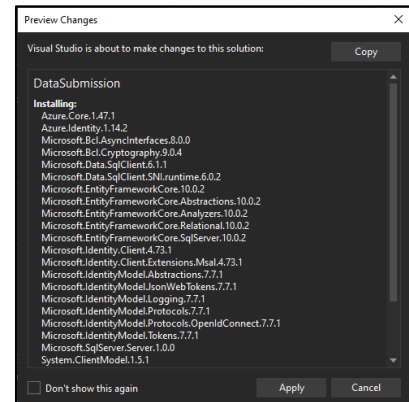
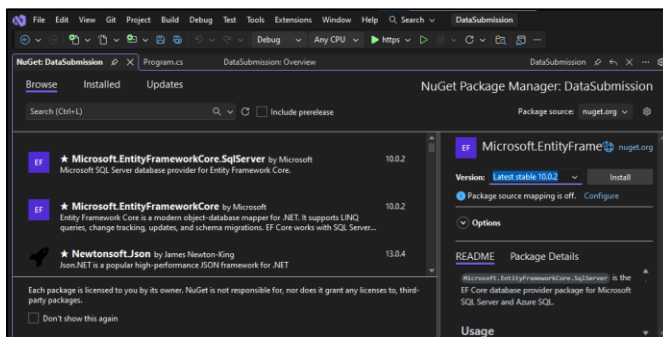
Install packages: Right click name of project "DataSubmission"



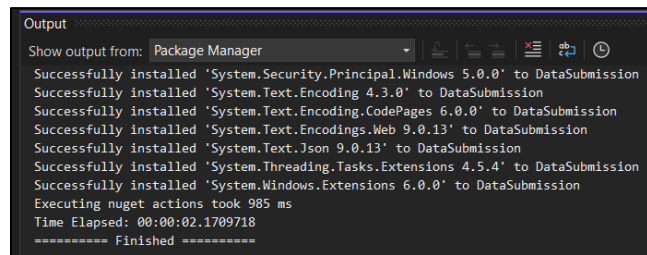
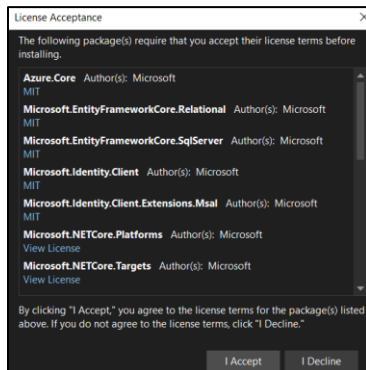
- Click on **Browse**.



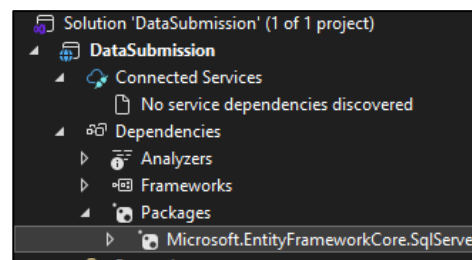
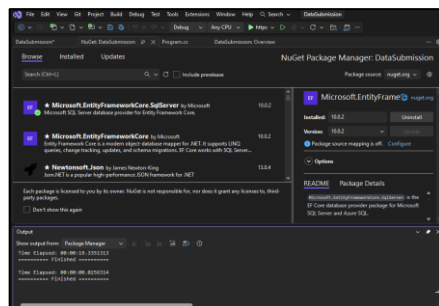
- Install first package **Microsoft.EntityFrameworkCore.SqlServer** by Microsoft & **apply**.



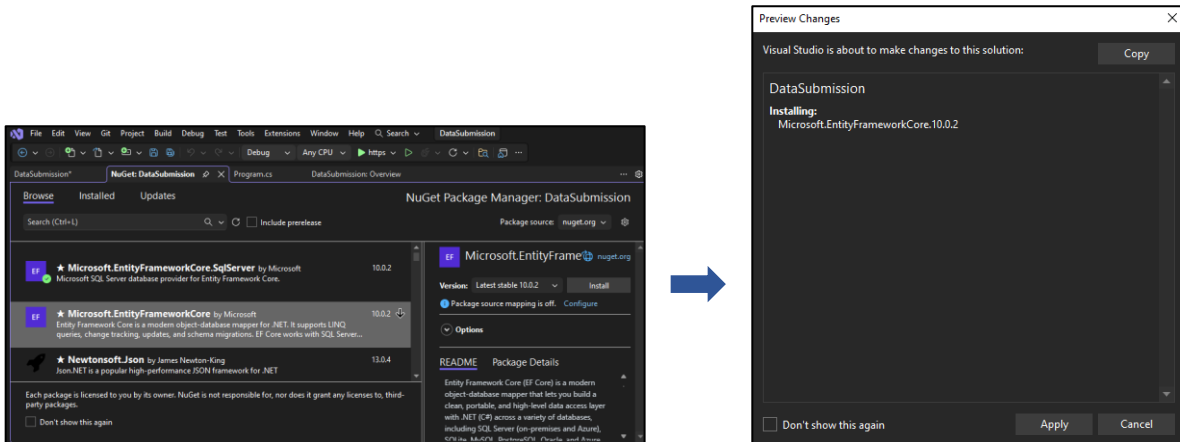
- Click **I Accept** then package successfully installed.



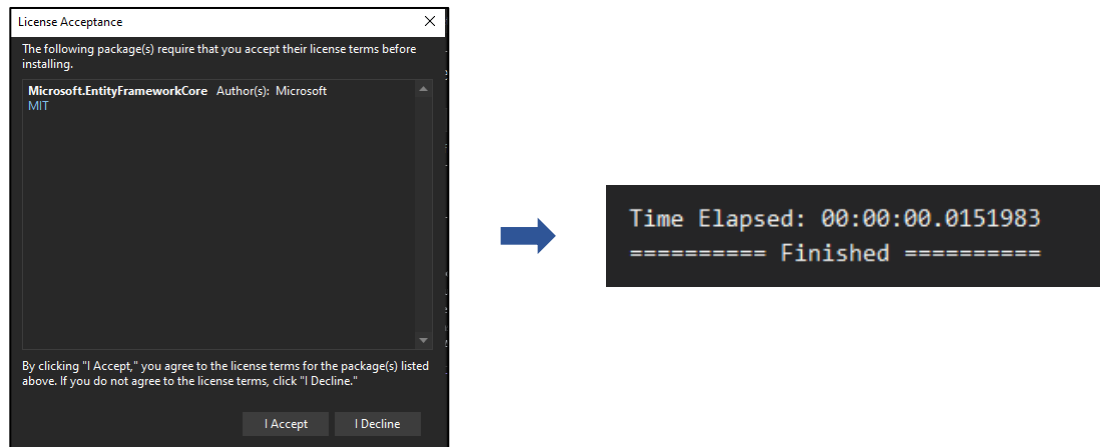
- The **Green icon** will appear on **Package** after installation Now check in **Dependencies** Package will be show if install completely.



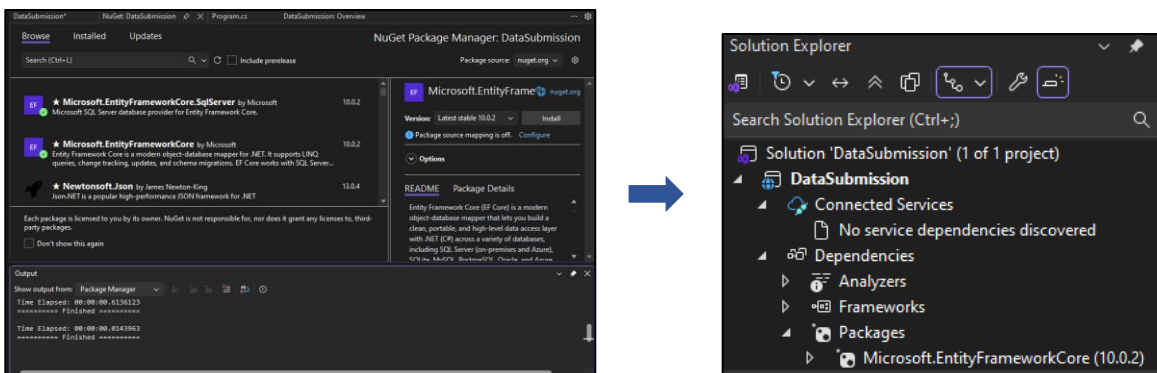
- Now Install **second package** & click **Apply**:



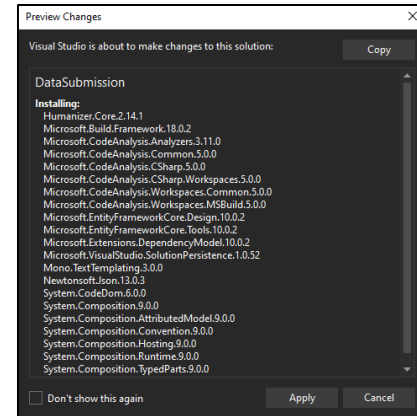
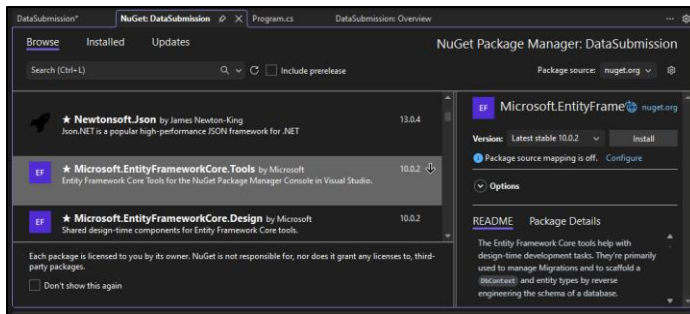
- Click **I Accept** then package successfully installed.



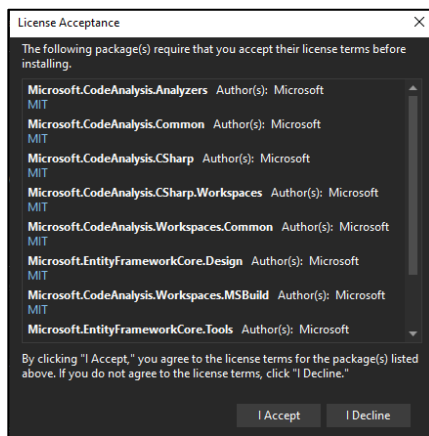
- The **Green icon** will appear on **Package** after installation Now check in **Dependencies** Package will be show if install completely.



- Now Install **third package** & click **Apply**:

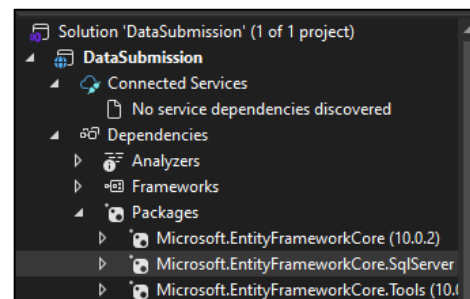
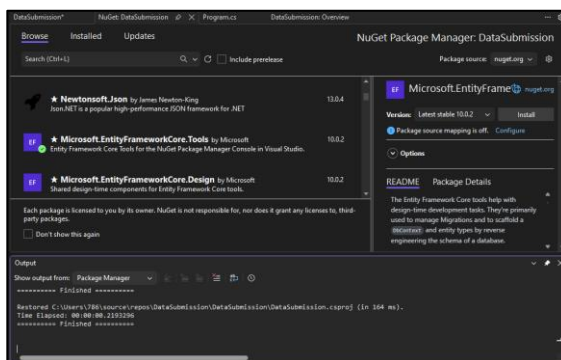


- Click **I Accept** then package successfully installed.

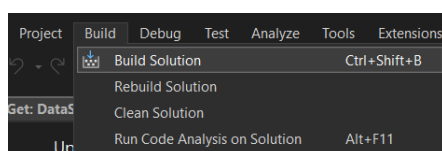


```
Restored C:\Users\USER\source\repos\DataSubmission\DataSubmission\DataSubmission.csproj (in 364 ms).
Time Elapsed: 00:00:00.9788406
***** Finished *****
```

- The **Green icon** will appear on **Package** after installation Now check in **Dependencies** Package will be show if install completely.



- After installing packages, In **Build** from Top, **Build Solution** no conflict occurs.

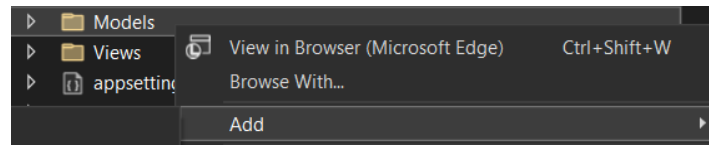


```
Output
Show output from: Build
Build started at 9:32 AM...
1>----- Build started: Project: DataSubmission, Configuration: Debug Any CPU -----
1>DataSubmission -> C:\Users\USER\source\repos\DataSubmission\DataSubmission\bin\Debug\net8.0\DataSubmission.dll
***** Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped *****
***** Build completed at 9:32 AM and took 20.503 seconds *****
```

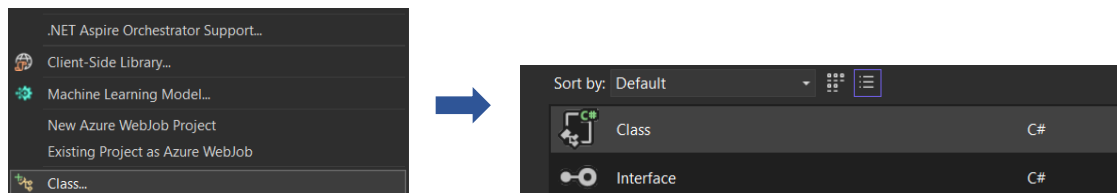
Step 2: Create Model Class

- Create a Models folder.
- Create class Student with properties: Id, Name, Email, Password, Gender, Age.
- Apply data annotations (Required, etc.)

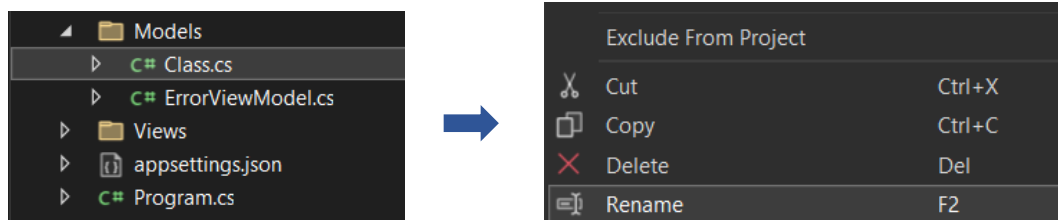
- **Create Model class:** Model class is the one which is named as the table name we want create for example student
- We don't create database directly in SQL server instead we use code first approach.
- For creating model class go to model and right click it then go to **ADD** & then **class**.



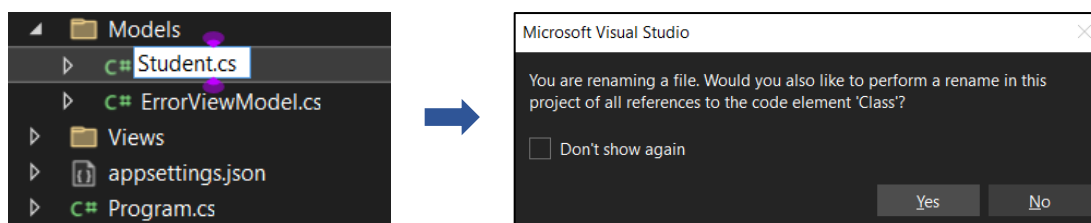
- After **ADD** choose class from list then click on class Class:



- **Class.cs** will create **Now Right click & Rename**



- Change Name **Student.cs** & **OK** for message.



- **Prop** then click on **tab**



- First Id **every table has id.**

```

1 namespace DataSubmission.Models
2 {
3     0 references
4     public class Student
5     {
6         0 references
7         public int id { get; set; }
8     }

```

- Creating Form:** Id, name, gender & password.

```

namespace DataSubmission.Models
{
    0 references
    public class Student
    {
        0 references
        public int id { get; set; }
        0 references
        public string name { get; set; }
        0 references
        public string Email { get; set; }
        0 references
        public string Password { get; set; }
        0 references
        public string Gender { get; set; }
        0 references
        public int age { get; set; }
    }
}

```

- Setting primary Key:** As we know each table has primary key in sql so make the id primary key. Key is in system.component.model. data annotations. Double click it now it show on top of code.

[key]

0 references

Key System.ComponentModel.DataAnnotations

Keyless Microsoft.EntityFrameworkCore

ConfigurationKeyName



```

1 using System.ComponentModel.DataAnnotations;
2
3 namespace DataSubmission.Models
4 {
5     0 references
6     public class Student

```

- Making required:** Making all things compulsory for it type [Required]

[re]

0 references

return

RegularExpression

Required

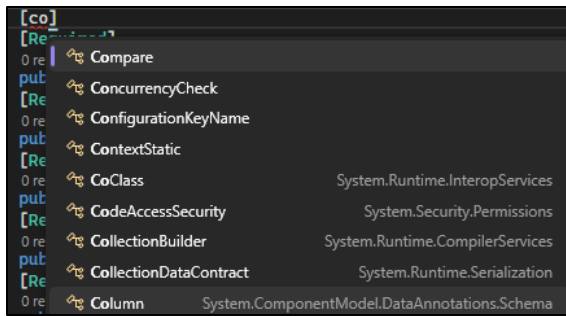


```

using System.ComponentModel.DataAnnotations;
namespace DataSubmission.Models
{
    0 references
    public class Student
    {
        [Key]
        0 references
        public int id { get; set; }
        [Required]
        0 references
        public string name { get; set; }
        [Required]
        0 references
        public string Email { get; set; }
        [Required]
        0 references
        public string Password { get; set; }
        [Required]
        0 references
        public string Gender { get; set; }
        [Required]
        0 references
        public int age { get; set; }
    }
}

```

- Setting Column:** If we want that name is not shown in table (column name) instead student name show and so on then we use[column] and set it's name and datatype.



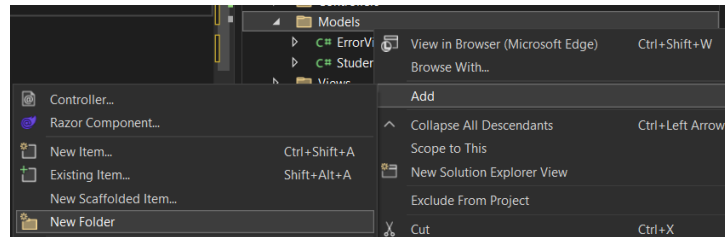
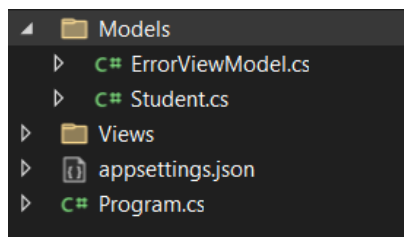
```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace DataSubmission.Models
{
    [Table("Student")]
    public class Student
    {
        [Key]
        public int id { get; set; }
        [Column("StudentName"), DataType(DataType.Text)]
        [Required]
        public string name { get; set; }
        [Column("StudentEmail"), DataType(DataType.EmailAddress)]
        [Required]
        public string Email { get; set; }
        [Column("StudentPassword"), DataType(DataType.Password)]
        [Required]
        public string Password { get; set; }
        [Column("StudentGender"), DataType(DataType.Text)]
        [Required]
        public string Gender { get; set; }
        [Required]
        public int age { get; set; }
    }
}
```

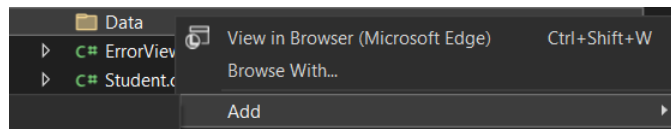
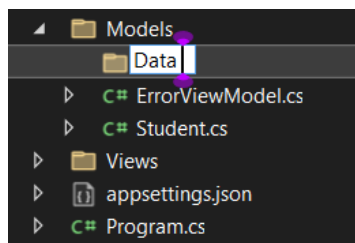
Step 3: Create DbContext Class

- Create class StudentDbContext inheriting from DbContext.
- Accept DbContextOptions in constructor.
- Add DbSet<Student> Students { get; set; }

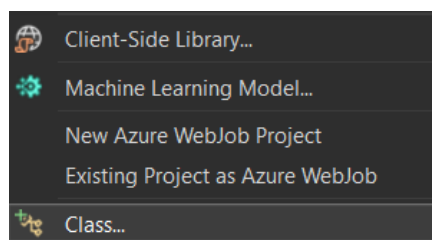
- **Context Class:** Model class and SQL server cannot directly connect with each other for their connection we have to use core entity framework which we installed previously and we have to create another db context class.
- For this right click model click on Add & then new folder.



- Rename & in **data** folder create **class**.

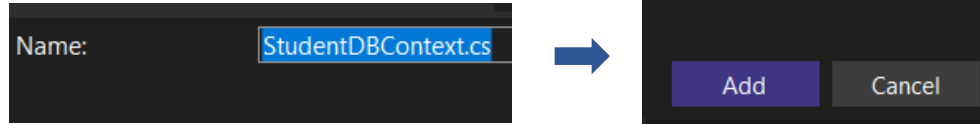


- Change class name from bottom bar:

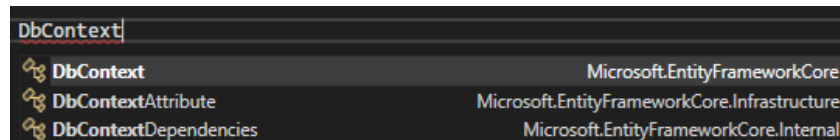
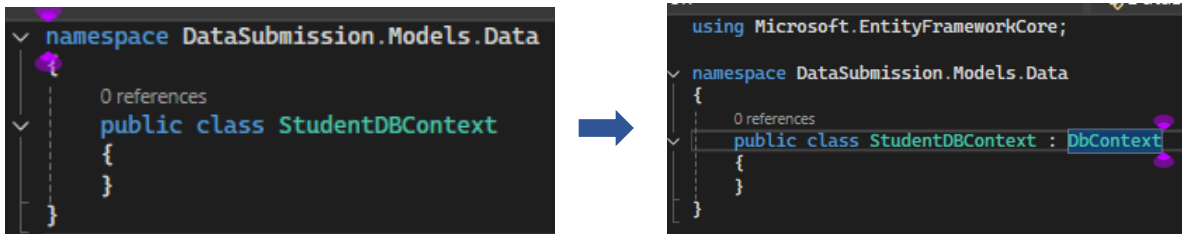


Name:

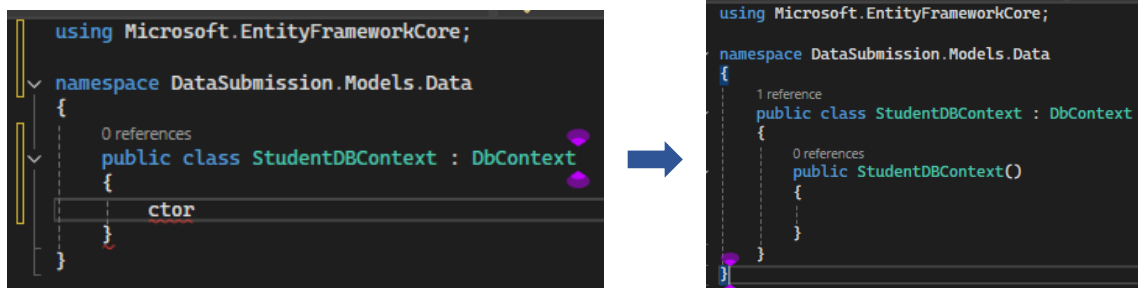
- Write Name like **StudentDbContext.cs** & **Add**



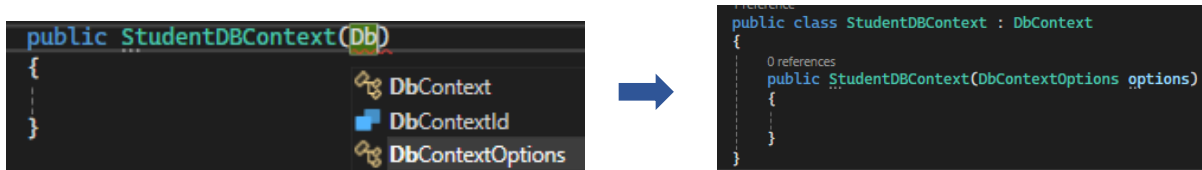
- Inherit** the class with actual already created **DbContext** class.



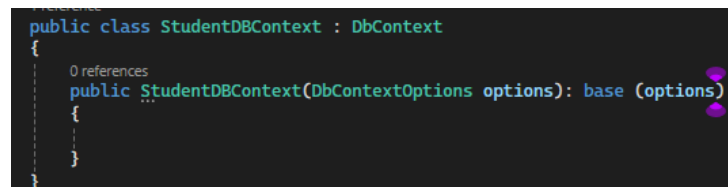
- Creating Constructor: type **ctor** and press **tab**.



- Give a parameter **option**.



- Inherit** from base class Student **DbContext**



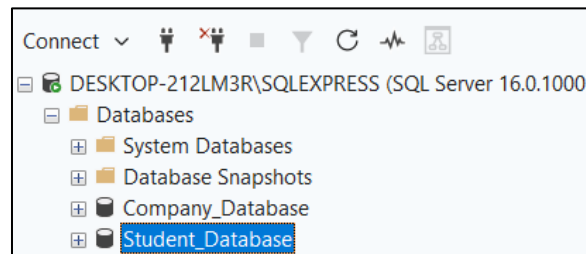
- Connect Model class column** with context class for this type **prop** press Tab and type **DbSet<Student> Student{get;set}.**

```
using Microsoft.EntityFrameworkCore;

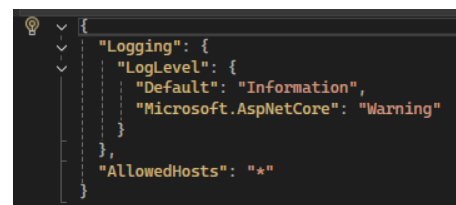
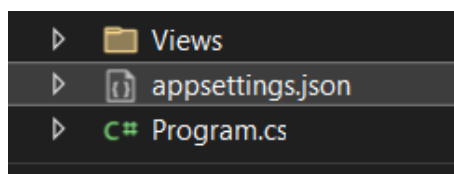
namespace DataSubmission.Models.Data
{
    1 reference
    public class StudentDBContext : DbContext
    {
        0 references
        public StudentDBContext(DbContextOptions options): base (options)
        {
        }
        0 references
        public DbSet<Student> Student { get; set; }
    }
}
```

Step 4: Add Connection String in appsettings.json
 - Add connection string with key DefaultConnection
 - "ConnectionStrings": {
 "DefaultConnection": "Server=your_server_name;Database=your_database_name;Trusted_Connection=True;TrustServerCertificate=True}

- **Student Database created:**



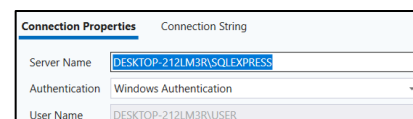
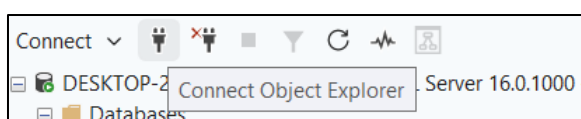
- **Add connection string** in **appsetttings.json**.
- Now we want connection of SQL server & the project we created for it go to solution explorer second last option app settion.json.



- "ConnectionStrings": { "DefaultConnection":
 "Server=your_server_name;Database=your_database_name;Trusted_Connection=True;Tr
 ustServerCertificate=True} **copy this path** to the .json file & **add your server name**.

Step 4: Add Connection String in appsettings.json
 - Add connection string with key DefaultConnection
 - "ConnectionStrings": {
 "DefaultConnection": "Server=your_server_name;Database=your database name;Trusted_Connection=True;TrustServerCertificate=True}

- For **server name** go to the SQL server **copy server name** and **paste it in .jsoon** file.





- Also **add your database name** like **DatasubmissionDB** now a database of this name with student table created automatically on SQL server.



```

{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "ConnectionStrings": {
    "dbcs": "Server=DESKTOP-212LM3R\\SQLEXPRESS;Database=DataSubmissionDB;Trusted_Connection=True;TrustServerCertificate=True"
  },
  "AllowedHosts": "*"
}

```

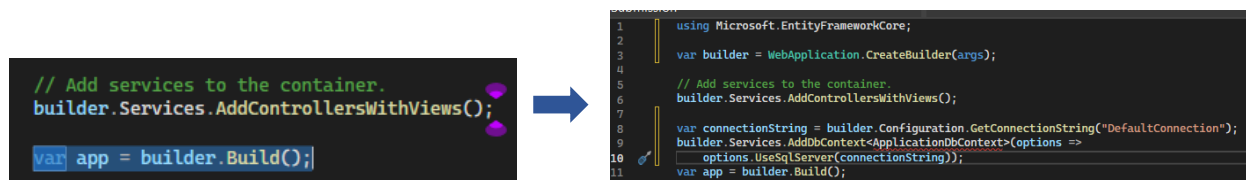
Step 5: Register Connection String in Program.cs

- Read connection string from configuration.
- Register DbContext using AddDbContext and SQL Server provider.
- `var connectionString = builder.Configuration.GetConnectionString("DefaultConnection");`
- `builder.Services.AddDbContext<ApplicationDbContext>(options =>`
- `options.UseSqlServer(connectionString));`

- When project run it direct comes in **program.cs** now we **register connection string in program.cs**
- For it **paste** `"var connectionString = builder.Configuration.GetConnectionString("DefaultConnection");`
`builder.Services.AddDbContext<ApplicationDbContext>(options =>`
`options.UseSqlServer(connectionString));"` in program.cs file change the database name with the created database name.

Step 5: Register Connection String in Program.cs

- Read connection string from configuration.
- Register DbContext using AddDbContext and SQL Server provider.
- `var connectionString = builder.Configuration.GetConnectionString("DefaultConnection");`
- `builder.Services.AddDbContext<ApplicationDbContext>(options =>`
- `options.UseSqlServer(connectionString));`



- In program.cs file change the database name with the created database name.

```
var connectionString = builder.Configuration.GetConnectionString("DefaultConnection");
builder.Services.AddDbContext<StudentDBContext>(options =>
    options.UseSqlServer(connectionString));
var app = builder.Build();
```

```
var connectionString = builder.Configuration.GetConnectionString("DefaultConnection");
builder.Services.AddDbContext<StudentDBContext>(options =>
    options.UseSqlServer(connectionString));
var app = builder.Build();
```

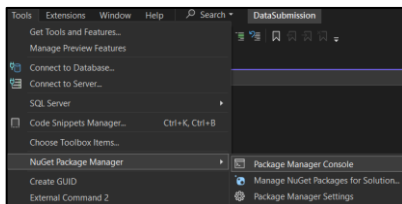
- Also **change default connection** with **dbcs** we named it in **.json file**

```
var connectionString = builder.Configuration.GetConnectionString("DefaultConnection");
builder.Services.AddDbContext<StudentDBContext>(options =>
    options.UseSqlServer(connectionString));
var app = builder.Build();
```

```
var connectionString = builder.Configuration.GetConnectionString("dbcs");
builder.Services.AddDbContext<StudentDBContext>(options =>
    options.UseSqlServer(connectionString));
var app = builder.Build();
```

Step 6: Migration & Update Database
 - Use Package Manager Console or CLI:
 - Add-Migration InitialCreate
 - Update-Database

- Migration and update database:** we do it at the time database & table will created in SQL.
- For this go to **Tools** in **nugget Package manager** go to **package manager console**. Type **Add Migration command:**



```
Package source: All
PM> Add-Migration FormSubmission
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM>
```

- Add Migration file:** Detail of file show (name, email, gender) & primary key etc.

```
1 using Microsoft.EntityFrameworkCore.Migrations;
2
3 #nullable disable
4
5 namespace DataSubmission.Migrations
6 {
7     /// <inheritdoc />
8     public partial class FormSubmission : Migration
9     {
10         /// <inheritdoc />
11         protected override void Up(MigrationBuilder migrationBuilder)
12         {
13             migrationBuilder.CreateTable(
14                 name: "Student",
15                 columns: table => new
16                 {
17                     id = table.Column<int>(type: "int", nullable: false)
18                         .Annotation("SqlServer:Identity", "1, 1"),
19                     StudentName = table.Column<string>(type: "nvarchar(max)", nullable: false),
20                     StudentEmail = table.Column<string>(type: "nvarchar(max)", nullable: false),
21                     StudentPassword = table.Column<string>(type: "nvarchar(max)", nullable: false),
22                     StudentGender = table.Column<string>(type: "nvarchar(max)", nullable: false),
23                     age = table.Column<int>(type: "int", nullable: false)
24                 },
25                 constraints: table =>
26                 {
27                     table.PrimaryKey("PK_Student", x => x.id);
28                 });
29         }
30     }
```

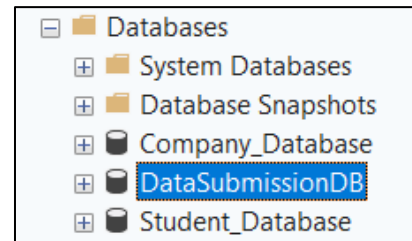
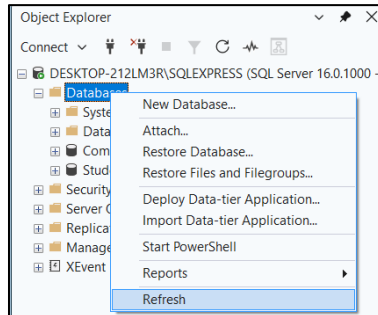
- **Second command:** Update command

```
PM: Update-Database
Build started...
Build succeeded.
Microsoft.EntityFrameworkCore.Database.Command[20101]
  Executed DbCommand (578ms) [Parameters=[], CommandType='Text', CommandTimeout='60']
  CREATE DATABASE [DataSubmissionDB];
Microsoft.EntityFrameworkCore.Database.Command[20101]
  Executed DbCommand (455ms) [Parameters=[], CommandType='Text', CommandTimeout='60']
  IF SERVERPROPERTY('engineedition') <> 5
  BEGIN
    ALTER DATABASE [DataSubmissionDB] SET READ_COMMITTED_SNAPSHOT ON;
  END;
Microsoft.EntityFrameworkCore.Database.Command[20101]
```

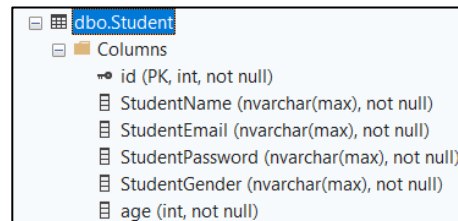
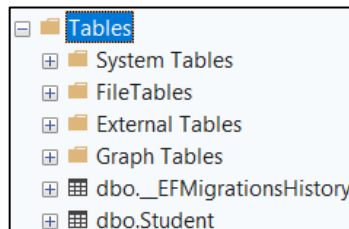


```
Microsoft.EntityFrameworkCore.Database.Command[20101]
  Executed DbCommand (48ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
  INSERT INTO [_EFMigrationsHistory] ([MigrationId], [ProductVersion])
  VALUES (N'20260223054936_FormSubmission', N'0.0.13');
Microsoft.EntityFrameworkCore.Database.Command[20101]
  Executed DbCommand (190ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
  DECLARE @result int;
  EXEC @result = sp_releaseapplock @Resource = '__EFMigrationsLock', @LockOwner = 'Session';
  SELECT @result
Done.
```

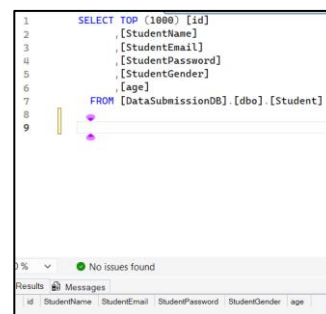
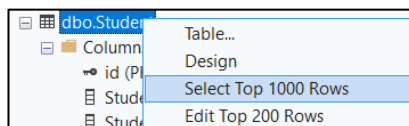
- Firstly **Refresh** then check whether the **FormSubmission** database with student table in SQL server is created or not.



- In **Tables**, All the **columns** we created in Modal class show here.



- Now check if there is values or not. For now no value show in it.

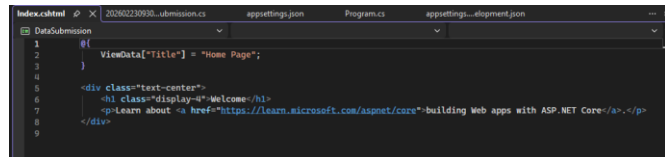
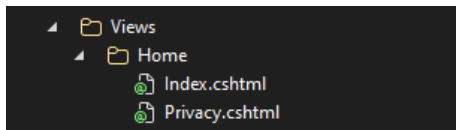


Step 7: Controller Injection and Create User Form Page (Create Template)

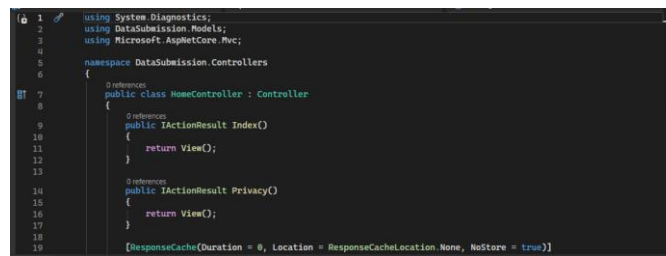
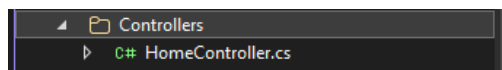
- Controller with constructor injection of DbContext.
- Scaffold Create Razor page for Student.
- Fields: Name, Email, Password, Gender, Age.

- Now this step, when the **user fills form enter data** such as **name email gender** etc & **submit** the form the data will show in above screen.

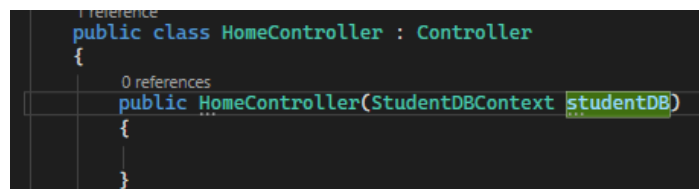
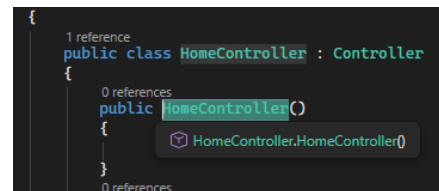
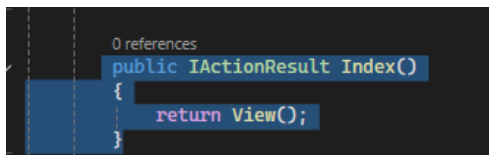
- Controller Injection & **Create User Form** Page (Create Template). Controller with constructor injection of **DbContext**. Now go to **Controller page** & make it's **connection** with database.
- User interact with index page** in views but this **index page is not directly contact** with database it is connected to Controller and controller is connected to database.



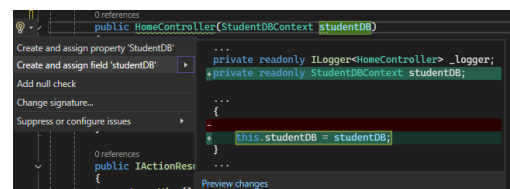
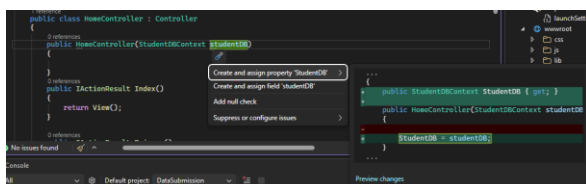
- Now go to **home controller.cs** page



- Now **make a constructor** on IsAction Result **Index()**. The constructor has same name of class & give it **parameter of the DB context** class and give a name to object **studentDB**



- Now point on **studentDB** press **Ctrl+.**, this screen appear. **Go to second option** Create and Assign field "studentDB".

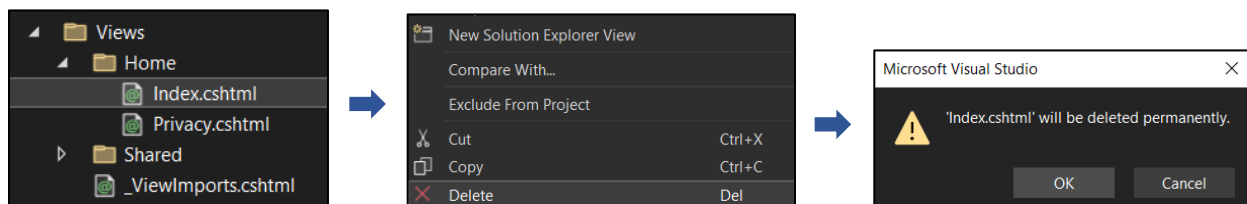


- `this.studentDB = studentDB;` now we created connection with database. This is called injection.

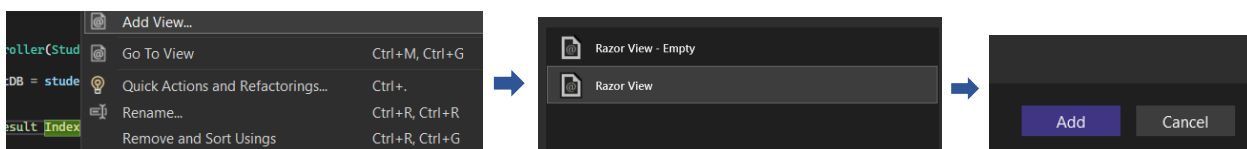
```
1 reference
public class HomeController : Controller
{
    private readonly StudentDBContext studentDB;

    0 references
    public HomeController(StudentDBContext studentDB)
    {
        this.studentDB = studentDB;
    }
}
```

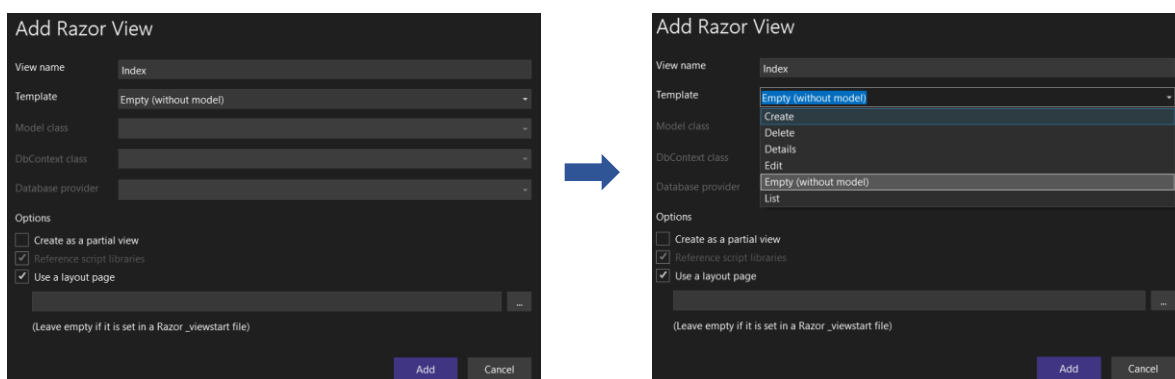
- Now we go to **user view** the **index.cshtml** & **delete** it then **OK**.



- Now we **want to create new view named index**. For this **right click Index** and **go to Add View**. Then **go to Razor View**. Razor view is **use when we want to insert c# code in html** file the razor engine then convert the c# code in html. Now **click on Add**.



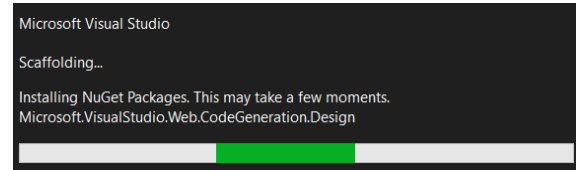
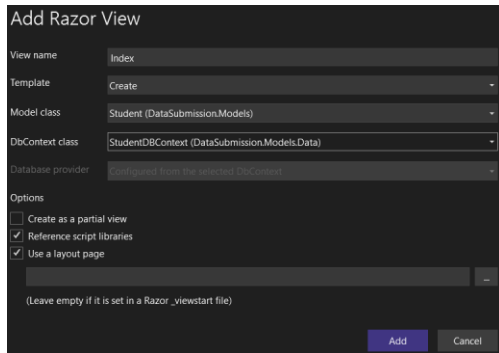
- Now this screen appear & **change the template** to **Create** due to it the table we created will automatically change to form.



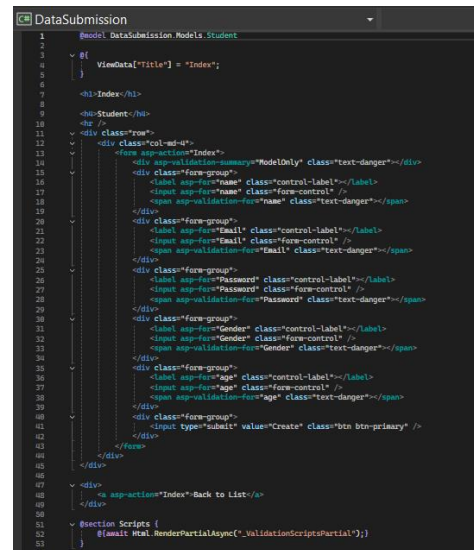
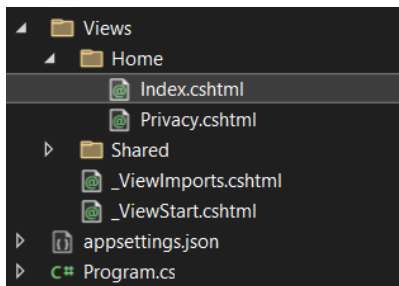
- Enter the **Modal** and **DB class name**.



- Click on **Add** Process start, now view file is creating.



- Form creation successful.** `<form asp-action="Index">` it means we want to run index action in controller.



Index

Student

name

Email

Password

Gender

age

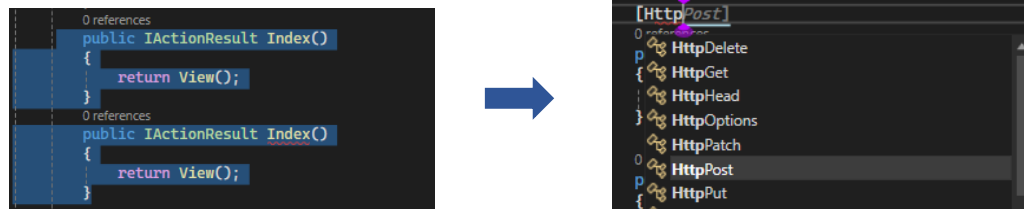
Create

[Back to List](#)

Step 8: HTTP POST Action to Submit Data

- [HttpPost] action accepting Student model.
- Validate ModelState.
- Save data using DbContext.

- **HTTP POST Action** to Submit Data(sending data from user to database)
- Now **the http post action to submit data** that when user enter data in above form it should submit. **To tell the controller about post action** go to **controller file** & write the **constructor** again & **above it write** [HTTP post]



```

0 references
public IActionResult Index()
{
    return View();
}

0 references
public IActionResult Index()
{
    return View();
}
    
```

→

```

[HttpPost]
0 references
public IActionResult Index()
{
    return View();
}

0 references
[HttpPost]
0 references
public IActionResult Index(Student std)
{
    return View();
}
    
```

- **Give modal class** & it's **object as parameter**. Now **when user enter data** & click on **Create it will store in std object**.



```

0 references
public IActionResult Index()
{
    return View();
}

[HttpPost]
0 references
public IActionResult Index()
{
    return View();
}
    
```

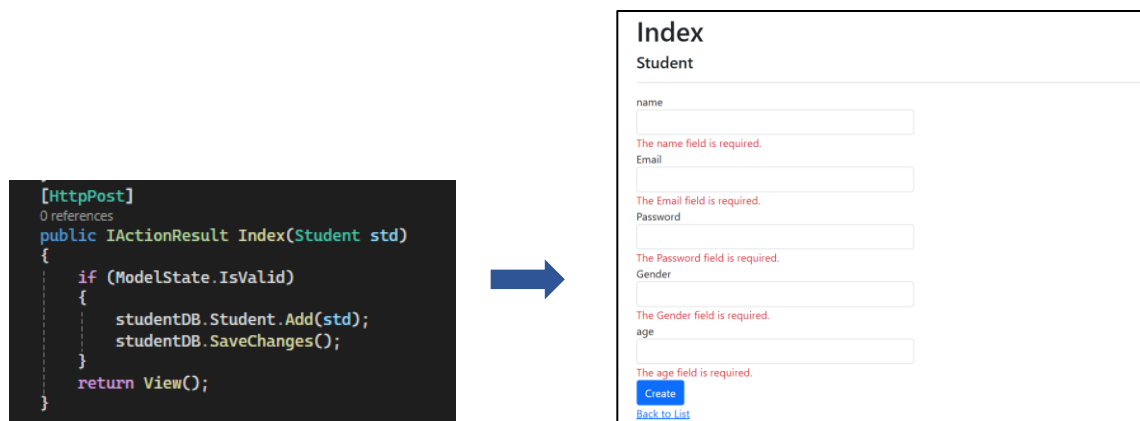
→

```

0 references
public IActionResult Index()
{
    return View();
}

[HttpPost]
0 references
public IActionResult Index(Student std)
{
    return View();
}
    
```

- Now **Validate ModelState** & **Save data** using **DbContext**.
- For this give condition in **Home Controller.cs**. HTTP post section that if all conditions in **Model class is valid then save changes** & **add the values in std object** in student DB to **which Modal class is connected**. Now the **data will save in SQL Server** in database.
- Now if **we don't enter all the fields** then **JavaScript validation** will apply.



```

[HttpPost]
0 references
public IActionResult Index(Student std)
{
    if (ModelState.IsValid)
    {
        studentDB.Student.Add(std);
        studentDB.SaveChanges();
    }
    return View();
}
    
```

→

Index
Student

name

The name field is required.

Email

The Email field is required.

Password

The Password field is required.

Gender

The Gender field is required.

age

The age field is required.

[Create](#)
[Back to List](#)

Step 9: Show Success Alert Message

- After successful submission, show alert: "Data successfully submitted" using TempData or JavaScript.

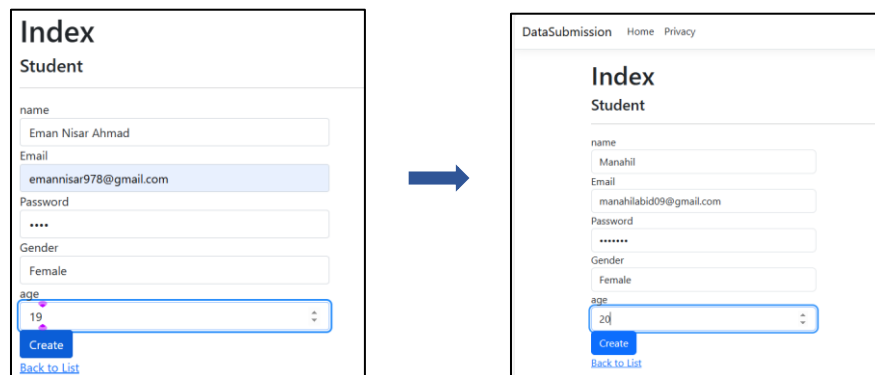
- Show **Success Alert Message**, After **successful submission**, show alert: "**Data successfully submitted**" use TempData or JavaScript. **TempData used to show the c#code in html.**

```
[HttpPost]
public IActionResult Index(Student std)
{
    if (ModelState.IsValid)
    {
        studentDB.Student.Add(std);
        studentDB.SaveChanges();
        TempData["successmessage"] = "Data Successfully submitted";
    }
    return View();
}
```

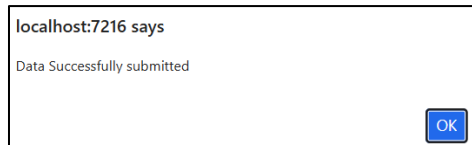
- Now to show it in html we use razor view syntax using @.
- After saving changes in program.cs file then this condition will apply.

```
@if (TempData["successmessage"] != null)
{
    <script>
        alert('TempData["successmessage"]');
    </script>
}
```

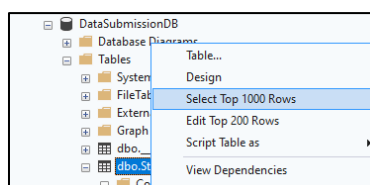
- **Now run** the form.



- For **Data Successfully Submitted alert** pop up click **OK**.



- Now check the data should be store in SQL sever. Right click student table and click on Edit top 1000 rows.



- Data appear successfully.

id	StudentName	StudentEmail	StudentPassword	StudentGender	age
1	Eman Nisar Ahmad	emannisar79@gmail.com	4343	Female	19

id	StudentName	StudentEmail	StudentPassword	StudentGender	age
1	Manahil	manahilabid09@gmail.com	1234567	Female	20

Step 10: Design the User Form

- For this *go to indexhtml.cs file & design the form as we want.*

CRUD operations:

- **Create:** previously we create a form now we want to form a create option for it so *rename* the *index.cshtml* file to *create.cshtml*.



- *Change the name in view & post method in Home Controller.cs* also.

```

0 references
public IActionResult Create()
{
    return View();
}
[HttpPost]
0 references
public IActionResult Create(Student std)
{

```

- Also *change the action in Create.cshtml* to Create which was Index previously & also *define method* which is post.

```

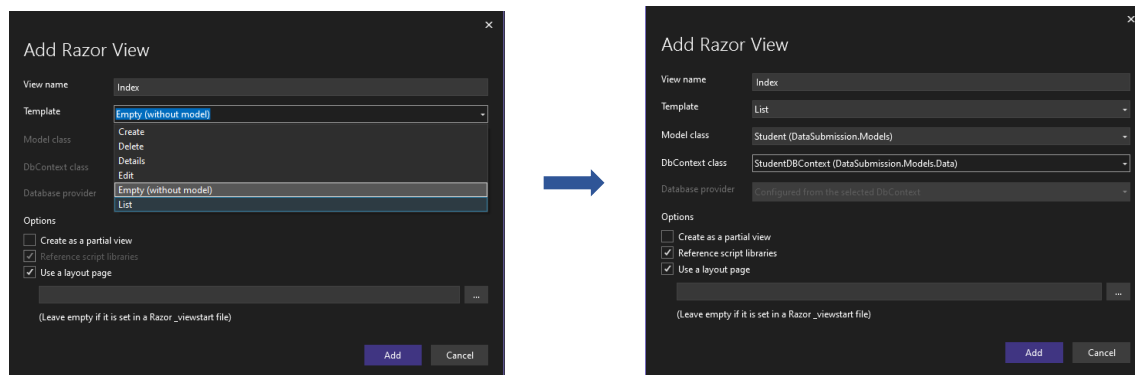
DataSubmission
1 @model DataSubmission.Models.Student
2
3 @{
4     ViewData["Title"] = "Index";
5 }
6
7 <h1>Index</h1>
8 <h4>Student</h4>
9 <hr />
10 <div class="row">
11     <div class="col-md-4">
12         <form asp-action="create" method="post">
13             <div asp-validation-summary="ModelOnly" class="text-danger"></div>
14             <div class="form-group">
15                 <label asp-for="name" class="control-label"></label>
16                 <input asp-for="name" class="form-control" />
17                 <span asp-validation-for="name" class="text-danger"></span>
18             </div>
19             <div class="form-group">
20                 <label asp-for="Email" class="control-label"></label>
21                 <input asp-for="Email" class="form-control" />

```

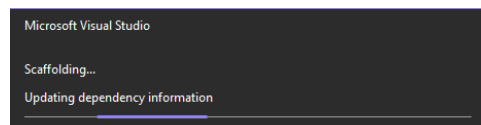
- **Index Page:** Now we want that the data in SQL server will show on index page. For this purpose again go to the **ActionResult Index** in **Home Controller.cs** right click it & click on **Add view** then go to **Razor View** & click on **Add**.



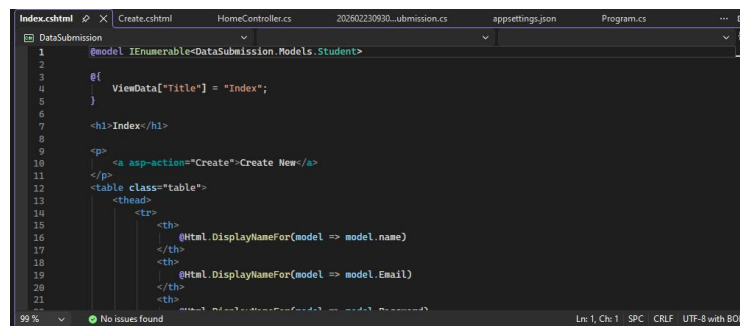
- **Choose List from Template** as we want to **Read** or **view** data in index page.
- **Choose List select DB & Modal class** & click on **Add**.



- **Creating** Index page.



- Index page **created**.



- **In Home Controller** we already created the index action now we want to show or view data we create a variable & in it **we take studentDB instance of context class & show the Model class Student data.**

```
public IActionResult Index()
{
    var stData = studentDB.Student.ToList();
    return View(stData);
}
```

- Now **Run** it. we can see **the Index page** is showing first & in it there is **the student data.**

</

- We can also see **create New. Click** on it form will show, Insert Data & **OK for Pop up.**

DataSubmission Home Privacy

Index

Student

name
Email
Password
Gender
age

Create Back to List

→

DataSubmission Home Privacy

Index

Student

name
Email
Password
Gender
age

Create Back to List

→

localhost:7075 says
TempData["successmessage"]

OK

- **Go back to list data** can be **view in Index page.**

DataSubmission

Home

Privacy

Index

Create New

name	Email	Password	Gender	age	
Manahil	manahilabd09@gmail.com	1234567	Female	20	Edit Details Delete
Eman	malika2808@gmail.com	1234	Female	19	Edit Details Delete

- Data is **also visible in database.**

SQLQuery3.s...OI\786 (82)

```

1 SELECT TOP (1000) [Id]
2     , [StudentName]
3     , [StudentEmail]
4     , [StudentPassword]
5     , [StudentGender]
6     , [age]
7 FROM [DataSubmissionDB].[dbo].[Student]
8

```

100 %
No issues found

Id	StudentName	StudentEmail	StudentPassword	StudentGender	age
1	Manahil	manahilabd09@gmail.com	1234567	Female	20
2	Eman	malika2808@gmail.com	1234	Female	19

- **Redirect on Home Page:** Now the next task is that when we enter data in form it redirects to Home page. For this **go to Home Controller.cs** & In it where we created message below it **remove return view** & **add redirect**.
- **Redirect ToAction("Index", "Home")** means redirect to Index page and controller will be Home.

```
public IActionResult Create(Student std)
{
    if (ModelState.IsValid)
    {
        studentDB.Student.Add(std);
        studentDB.SaveChanges();
        TempData["successmessage"] = "Data Successfully submitted";
    }
    return View();
}
```



```
if (ModelState.IsValid)
{
    studentDB.Student.Add(std);
    studentDB.SaveChanges();
    TempData["successmessage"] = "Data Successfully submitted";
}
RedirectToAction("Index", "Home");
}
```

- In case **if there is issue in redirecting** then it return view.

```
[HttpPost]
0 references
public IActionResult Create(Student std)
{
    if (ModelState.IsValid)
    {
        studentDB.Student.Add(std);
        studentDB.SaveChanges();
        //TempData["successmessage"] = "Data Successfully submitted";
        return RedirectToAction("Index", "Home");
    }
    return View(std);
}
```

- Now **run** & **again insert data** by clicking **Create new it will redirect** to Home page.

DataSubmission Home Privacy

Index

Student

name
Nimra

Email
Nimra@gmail.com

Password

Gender
Female

age
20

[Create](#)

[Back to List](#)



DataSubmission Home Privacy

Index

[Create New](#)

name	Email	Password	Gender	age
Manahil	manahilabid09@gmail.com	1234567	Female	20
Eman	maliks2808@gmail.com	1234	Female	19
Nimra	Nimra@gmail.com	56789	Female	20

[Edit](#) | [Details](#) | [Delete](#)

- **Details:** If we want to show details for example id which is not visible here but in database id is also visible. For it **go to index.cshtml** & in **table section add Display** For loop for id.

```
<table class="table">
<thead>
<tr>
<th>
@Html.DisplayNameFor(model => model.Id)
</th>
<th>
@Html.DisplayNameFor(model => model.name)
</th>
<th>
@Html.DisplayNameFor(model => model.Email)
</th>

```



```
@foreach (var item in Model) {
<tr>
<td>
@Html.DisplayFor(modelItem => item.Id)
</td>
<td>
@Html.DisplayFor(modelItem => item.name)
</td>
<td>
@Html.DisplayFor(modelItem => item.Email)

```

Index

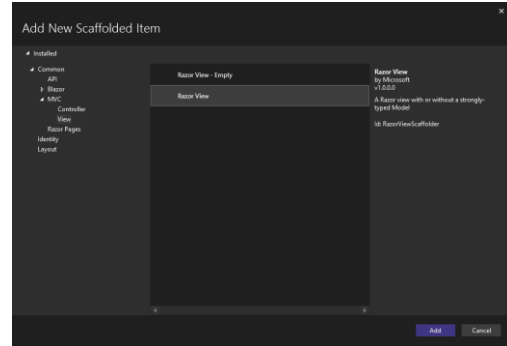
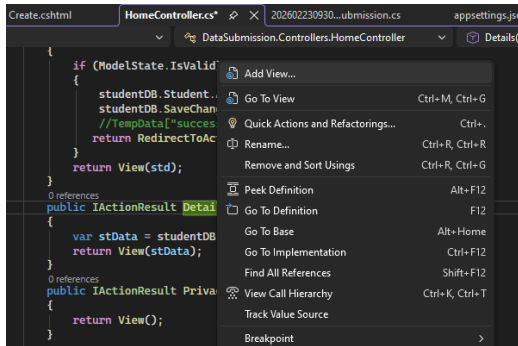
[Create New](#)

	Id	name	Email	Password	Gender	age
1	Manahil	manahilabid09@gmail.com	1234567	Female	20	Edit Details Delete
2	Eman	maliks2808@gmail.com	1234	Female	19	Edit Details Delete
3	Nimra	Nimra@gmail.com	56789	Female	20	Edit Details Delete

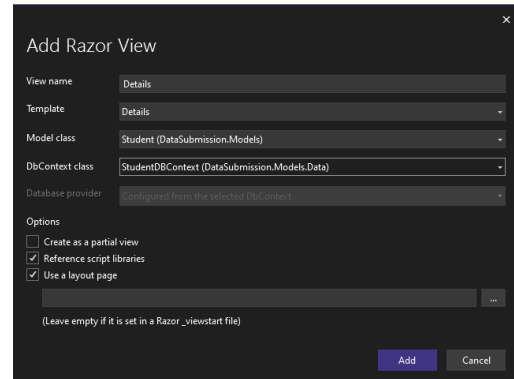
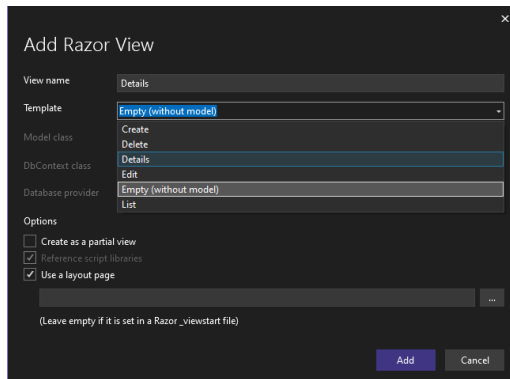
- Now to create detail page **go to Home controller.cs** create a Details action **& give Id as parameter.**

```
public IActionResult Details(int Id)
{
    var stData = studentDB.Student.ToList();
    return View(stData);
}
```

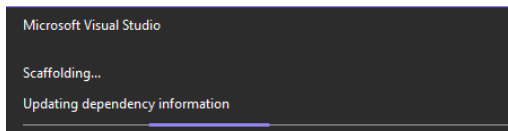
- Again create it's page **right click the Details** & click on **Add view then Razor View** & Click on **Add.**



- Select Details template** choose **Model & DB classes** and click on **Add.**



- Creating** page.



```
1 @model DataSubmission.Models.Student
2
3 @{
4     ViewData["Title"] = "Details";
5 }
6
7 <h1>Details</h1>
8
9 <div>
10     <h2>Student</h2>
11     <hr />
12     <table class="row">
13         <tr>
14             <td class="col-sm-2">
15                 @Html.DisplayNameFor(model => model.name)
16             </td>
17             <td class="col-sm-10">
18                 @Html.DisplayFor(model => model.name)
19             </td>
20         </tr>
21         <tr>
22             <td class="col-sm-2">
23                 @Html.DisplayNameFor(model => model.Email)
24             </td>
```

- **Add Id** list in it.

```

9      <div>
10      <h4>Student</h4>
11      <hr />
12      <dl class="row">
13      <dt class="col-sm-2">
14          @Html.DisplayNameFor(model => model.Id)
15      </dt>
16      <dd class="col-sm-10">
17          @Html.DisplayFor(model => model.Id)
18      </dd>

```

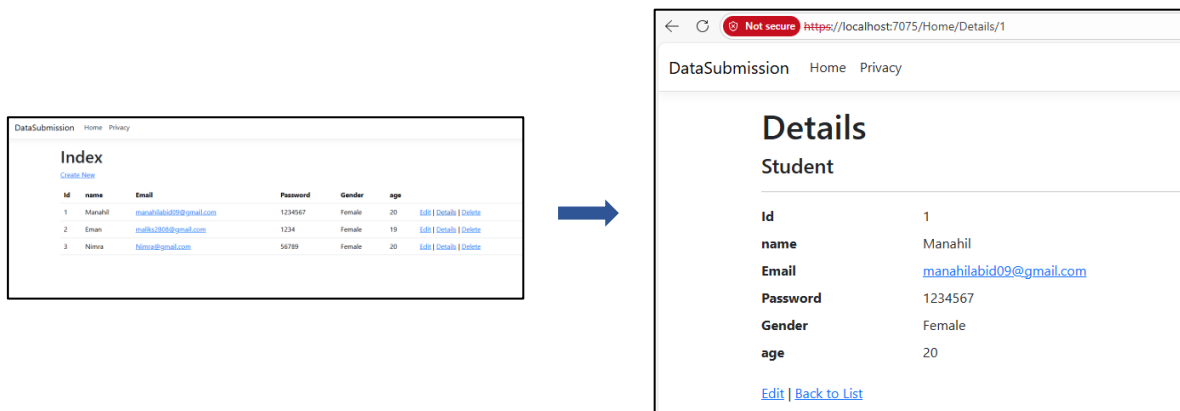
- Now in **Home Controller.cs** add **FirstOrDefault** in variable **stData**. It **matches** the **parameters id with database Id** if it found it **show details** otherwise if **condition run** which is “**if Id==null**” then **Not Found**.

```

38      0 references
39      public IActionResult Details(int Id)
40      {
41          var stData = studentDB.Student.FirstOrDefault(x=>x.Id==Id);
42          if(stData == null)
43          {
44              NotFound();
45          }
46          return View(stData);

```

- Click on Details now the Details page is visible and it is taking Id as parameter.



The Index page displays a table with the following data:

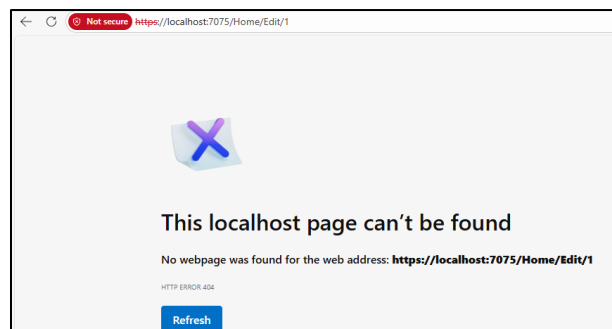
Id	name	Email	Password	Gender	age	
1	Manahil	manahilabid09@gmail.com	1234567	Female	20	Edit Details Delete
2	Eman	emank200@gmail.com	1234	Female	19	Edit Details Delete
3	Nimra	Nimra@gmail.com	56789	Female	20	Edit Details Delete

The Details page shows the information for the student with Id 1:

Details
Student

Id: 1
name: Manahil
Email: manahilabid09@gmail.com
Password: 1234567
Gender: Female
age: 20
[Edit](#) | [Back to List](#)

- **Edit:** For now edit page **cannot found** as it is **not created yet**.



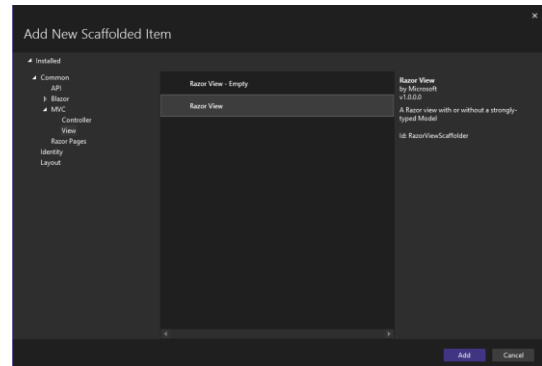
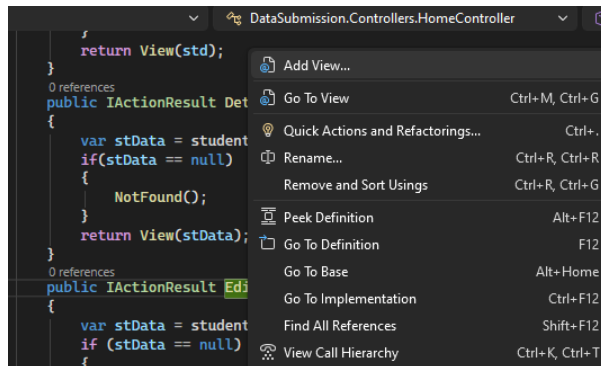
- For Edit page **go to Home Controller.cs** create Edit action & **give Id as parameter** now in edit one it **finds the values** in database using Id. If it **finds data then return** it otherwise show **not found**.

```

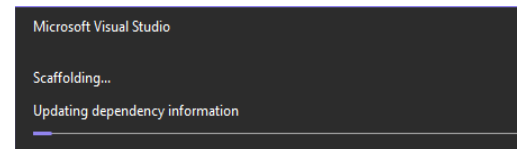
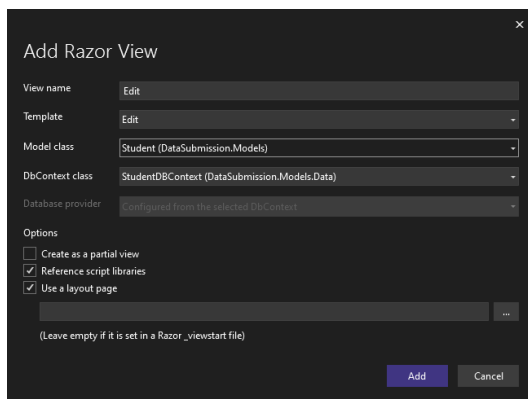
0 references
public IActionResult Edit(int Id)
{
    var stdData = studentDB.Student.Find(Id);
    if (stdData == null)
    {
        NotFound();
    }
    return View(stdData);
}

```

- Again create Edit page **right click the Edit** & click on **Add view then Razor View** & Click on **Add**.



- Select **Edit template** choose **Model and DB classes** and click on **Add**. Page **Creating** in process & **created**.

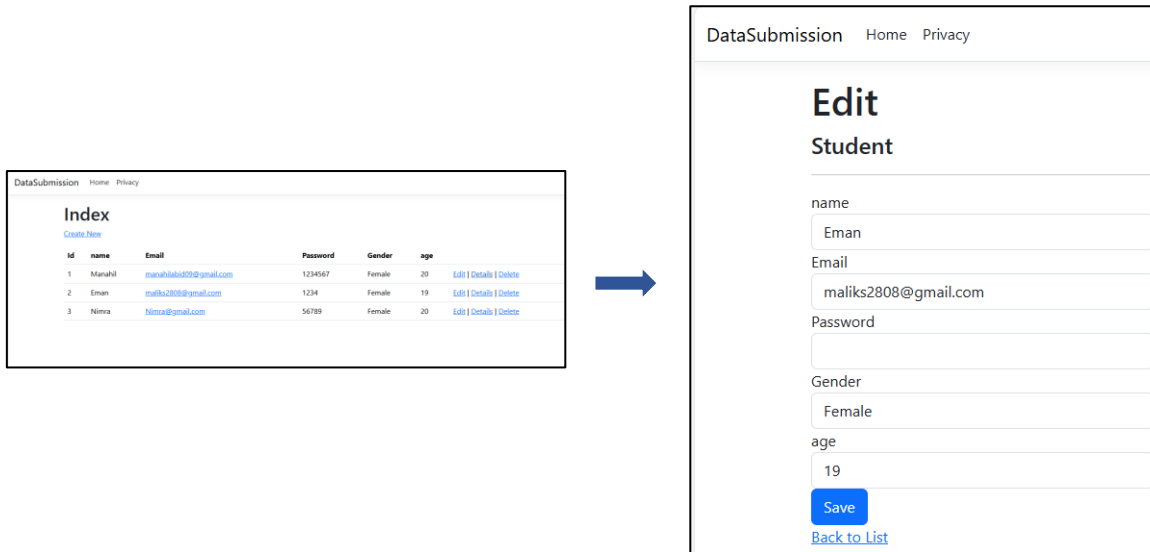


```

Edit.cshtml | Details.cshtml | Index.cshtml | Create.cshtml | HomeController.cs
1 @model DataSubmission.Models.Student
2
3 @{
4     ViewData["Title"] = "Edit";
5 }
6
7 <h1>Edit</h1>
8
9 <h4>Student</h4>
10 <hr />
11 <div class="row">
12     <div class="col-md-4">
13         <form asp-action="Edit">
14             <div asp-validation-summary="ModelOnly" class="text-danger"></div>
15             <input type="hidden" asp-for="Id" />
16             <div class="form-group">
17                 <label asp-for="name" class="control-label"></label>
18                 <input asp-for="name" class="form-control" />
19                 <span asp-validation-for="name" class="text-danger"></span>
20             </div>
21             <div class="form-group">
22                 <input type="text" asp-for="Email" class="form-control" />
23             </div>
24             <div class="form-group">
25                 <input type="text" asp-for="Phone" class="form-control" />
26             </div>
27             <div class="form-group">
28                 <input type="text" asp-for="Address" class="form-control" />
29             </div>
30             <div class="form-group">
31                 <input type="text" asp-for="City" class="form-control" />
32             </div>
33             <div class="form-group">
34                 <input type="text" asp-for="State" class="form-control" />
35             </div>
36             <div class="form-group">
37                 <input type="text" asp-for="Zip" class="form-control" />
38             </div>
39             <div class="form-group">
40                 <input type="text" asp-for="Country" class="form-control" />
41             </div>
42             <div class="form-group">
43                 <input type="text" asp-for="Password" class="form-control" />
44             </div>
45             <div class="form-group">
46                 <input type="text" asp-for="ConfirmPassword" class="form-control" />
47             </div>
48             <div class="form-group">
49                 <input type="button" value="Save" class="btn btn-primary" />
50             </div>
51         </form>
52     </div>
53 </div>

```

- Click on **Edit**. Now **Edit page is visible**.



The Index page shows a table with the following data:

Id	name	Email	Password	Gender	age	
1	Marahil	marahil609@gmail.com	1234567	Female	20	Edit Details Delete
2	Eman	maliks2808@gmail.com	1234	Female	19	Edit Details Delete
3	Nimra	Nimra@gmail.com	56789	Female	20	Edit Details Delete

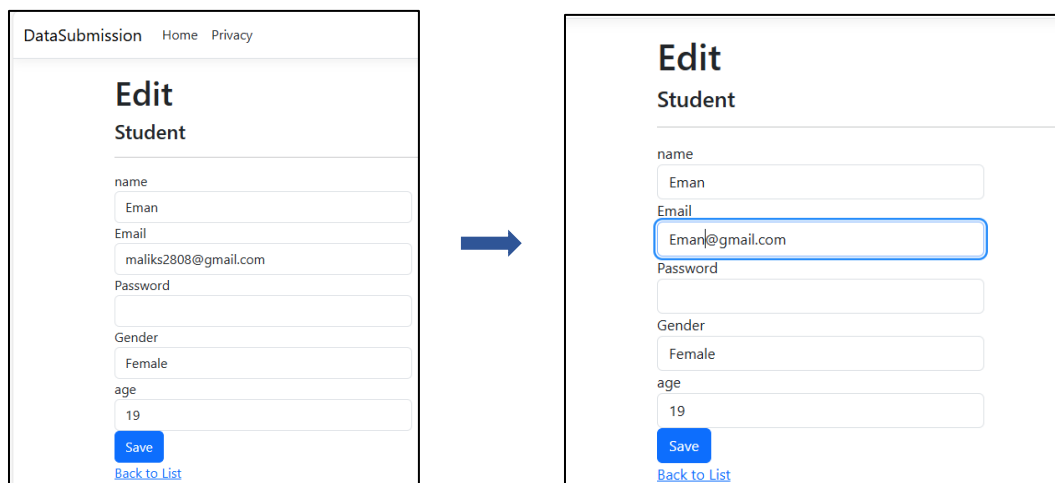
The Edit page for Student Eman shows the following form fields:

- name: Eman
- Email: maliks2808@gmail.com
- Password:
- Gender: Female
- age: 19
- Save button
- [Back to List](#)

- Next step is **the editing process** that if we **edit anything** in this edit page it will also save in database. For this again **go to Home Controller.cs** we create **Http Post action** of Edit in which **we use update** that if the data is **Valid** then **update it with previous one**.

```
[HttpPost]
0 references
public IActionResult Edit(Student std)
{
    if (ModelState.IsValid)
    {
        studentDB.Student.Update(std);
        studentDB.SaveChanges();
        // TempData["successmessage"] = "Data Successfully submitted";
        return RedirectToAction("Index", "Home");
    }
    return View(std);
}
```

- Now we **update the email** of the student & click on **save it will redirect to home page & changes will save & visible in both database & Index page**.



The Edit page for Student Eman shows the following form fields:

- name: Eman
- Email: Eman@gmail.com
- Password:
- Gender: Female
- age: 19
- Save button
- [Back to List](#)

- Now we can see **updated Email**. **Email is updated in database** also.

DataSubmission Home Privacy

Index

[Create New](#)

Id	name	Email	Password	Gender	age	
1	Manahil	manahilabid09@gmail.com	1234567	Female	20	Edit Details Delete
2	Eman	Eman2008@gmail.com	1234	Female	19	Edit Details Delete
3	Nimra	Nimra@gmail.com	56789	Female	20	Edit Details Delete

SQLQuery4.s...01\786 (85) 100 % No issues found

```

1 SELECT TOP (1000) [Id]
2     ,[StudentName]
3     ,[StudentEmail]
4     ,[StudentPassword]
5     ,[StudentGender]
6     ,[age]
7 FROM [DataSubmissionDB].[dbo].[Student]
8

```

	Id	StudentName	StudentEmail	StudentPassword	StudentGender	age
1	1	Manahil	manahilabid09@gmail.com	1234567	Female	20
2	2	Eman	Eman2008@gmail.com	1234	Female	19
3	3	Nimra	Nimra@gmail.com	56789	Female	20

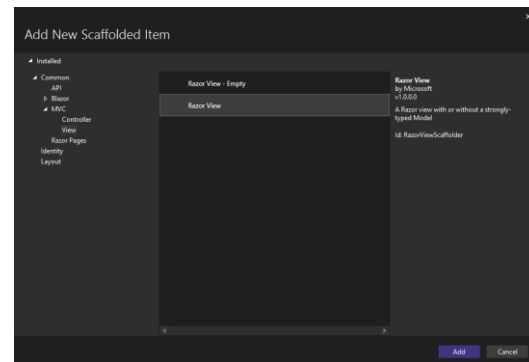
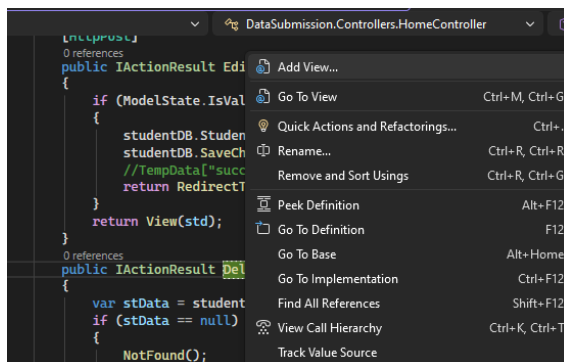
- **Delete:** If we want to delete any student data. For his again create an action in Home Controller.cs.

```

0 references
public IActionResult Delete(int Id)
{
    var stData = studentDB.Student.FirstOrDefault(x => x.Id == Id);
    if (stData == null)
    {
        NotFound();
    }
    return View(stData);
}

```

- Again create Delete page **right click the Delete** & **click on Add view** then **Razor View** & Click on **Add**.



- Select **Delete template** choose **Model and DB classes** & click on **Add**.

Add Razor View

View name: Delete

Template: Delete

Model class: Student (DataSubmission.Models)

DbContext class: StudentDbContext (DataSubmission.Models.Data)

Database provider: Configured from the selected DbContext

Options

☐ Create as a partial view

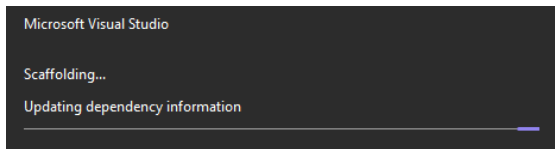
☒ Reference script libraries

☒ Use a layout page

(Leave empty if it is set in a Razor _viewstart file)

Add Cancel

- **Creating page** in process then **created**.



```

Delete.cshtml | Edit.cshtml | Details.cshtml | Index.cshtml | Create.cs
1 @model DataSubmission.Models.Student
2
3 @{
4     ViewData["Title"] = "Delete";
5 }
6
7 <h1>Delete</h1>
8
9 <h3>Are you sure you want to delete this?</h3>
10 <div>
11     <h4>Student</h4>
12     <hr />
13     <dl class="row">
14         <dt class="col-sm-2">
15             @Html.DisplayNameFor(model => model.name)
16         </dt>
17         <dd class="col-sm-10">
18             @Html.DisplayFor(model => model.name)
19         </dd>
20         <dt class="col-sm-2">
21             @Html.DisplayNameFor(model => model.Email)
22         </dt>
23         <dd class="col-sm-10">
24             @Html.DisplayFor(model => model.Email)
25         </dd>
26     </dl>
27 
```

- Now **we want that first the data show on page** then **Delete**. For this **go to Home Controller.cs** add **FirstOrDefault** in variable **stData**. It **matches the parameters id with database Id** if it **found it show details** before deletion otherwise if condition run which is **“if Id==null”** then **Not Found**. Click on **Delete**. It will take **id as parameter**.

Id	name	Email	Password	Gender	age
1	Manahil	manahilab02@gmail.com	1234567	Female	20
2	Eman	Emam2018@gmail.com	1234	Female	19
3	Nimra	Nimra@gmail.com	56789	Female	20



Not secure https://localhost:7075/Home/Delete/3

DataSubmission Home Privacy

Delete

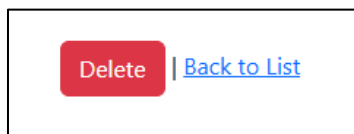
Are you sure you want to delete this?

Student

name	Nimra
Email	Nimra@gmail.com
Password	56789
Gender	Female
age	20

[Delete](#) | [Back to List](#)

- Now working on the **Delete button**. For this run **Http post function** for it in **Home Controller.cs**. Give it Action Name **“Delete”** so that **post action** will run **after delete action**. In Post action it will **check if the data is not Null** then **remove** the Model class student data & **save changes**.



```

77 [HttpPost, ActionName("Delete")]
78 0 references
79 public IActionResult DeleteData(int Id)
80 {
81     var stData = studentDB.Student.FirstOrDefault(x => x.Id == Id);
82     if (stData != null)
83     {
84         studentDB.Student.Remove(stData);
85     }
86     studentDB.SaveChanges();
87     return RedirectToAction("Index", "Home");
88 }

```

- Now **click on delete** now it will **redirect to Index page** & now **student 3 data is deleted**.

DataSubmission Home Privacy

Index

[Create New](#)

Id	name	Email	Password	Gender	age
1	Manahil	manahilabid09@gmail.com	1234567	Female	20
2	Eman	Eman2008@gmail.com	1234	Female	19
3	Nimra	Nimra@gmail.com	56789	Female	20

Not secure https://localhost:7075/Home/Delete/3

DataSubmission Home Privacy

Delete

Are you sure you want to delete this?

Student

name	Nimra
Email	Nimra@gmail.com
Password	56789
Gender	Female
age	20

[Delete](#) | [Back to List](#)

DataSubmission Home Privacy

Index

[Create New](#)

Id	name	Email	Password	Gender	age
1	Manahil	manahilabid09@gmail.com	1234567	Female	20
2	Eman	Eman2008@gmail.com	1234	Female	19

- **Data deleted** in **database** also.

100 % ✓ No issues found

Results Messages

	Id	StudentName	StudentEmail	StudentPassword	StudentGender	age
1	1	Manahil	manahilabid09@gmail.com	1234567	Female	20
2	2	Eman	Eman2008@gmail.com	1234	Female	19

- **Email Validation**: Make sure no student enter duplicate email.
- For this **update the Create & Edit post actions** in **Home controller.cs** & **add email validation in it**. Create **post action update**:

```
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Create(Student std)
{
    // Check if the email already exists
    if (studentDB.Student.Any(s => s.Email == std.Email))
    {
        // Add error to ModelState
        ModelState.AddModelError("Email", "This email already exists. Please use a different email.");
    }

    if (ModelState.IsValid)
    {
        studentDB.Student.Add(std);
        studentDB.SaveChanges();
        TempData["successmessage"] = "Student added successfully!";
        return RedirectToAction("Index");
    }
}
```

Create New Student

name

Nimra

Email

Eman@gmail.com

This email already exists. Please use a different email.

Password

Enter Password

Gender

Female

age

20

[Back](#) [Create Student](#)

- **Edit** post action update:

```
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Edit(Student std)
{
    // Check if email exists for another student
    if (studentDB.Student.Any(s => s.Email == std.Email && s.Id != std.Id))
    {
        ModelState.AddModelError("Email", "This email already exists. Please use a different email.");
    }

    if (ModelState.IsValid)
    {
        studentDB.Student.Update(std);
        studentDB.SaveChanges();
        TempData["successmessage"] = "Student updated successfully!";
        return RedirectToAction("Index");
    }
}
```

Edit Student

name

Eman

Email

manahilabid09@gmail.com

This email already exists. Please use a different email.

Password

Gender

Female

age

20

[Back](#) [Update Student](#)

- **Designing and alert**: For this go to index, cshtml file. Design the table tag.

```
<table class="table table-dark table-active table-bordered table-hover">
  <thead>
    <tr>
```

DataSubmission Home Privacy

Index

[Create New](#)

Id	name	Email	Password	Gender	age	
1	Manahil	manahilabid09@gmail.com	1234567	Female	20	Edit Details Delete
2	Eman	Eman2008@gmail.com	1234	Female	19	Edit Details Delete

• Index *page designing*:

```
@model IEnumerable<DataSubmission.Models.Student>
@{
    ViewData["Title"] = "Students List";
}

<div class="container mt-4">
    <div class="card shadow-lg">
        <div class="card-header bg-primary text-white d-flex justify-content-between align-items-center">
            <h3>Students List</h3>
            <a asp-action="Create" class="btn btn-light btn-sm"> Add New Student</a>
        </div>
        <div class="card-body">
            <table class="table table-bordered table-hover table-striped text-center">
                <thead class="table-dark">
                    <tr>
```

```
<tr>
    <th>Id</th>
    <th>Name</th>
    <th>Email</th>
    <th>Password</th>
    <th>Gender</th>
    <th>Age</th>
    <th>Actions</th>
</tr>
</thead>
<tbody>
    @foreach (var item in Model)
    {
        <tr>
            <td>@item.Id</td>
            <td>@item.name</td>
            <td>@item.Email</td>
            <td>@item.Password</td>
            <td>@item.Gender</td>
```

```
<td>
                <a asp-action="Edit" asp-route-id="@item.Id" class="btn btn-warning btn-sm">Edit</a>
                <a asp-action="Details" asp-route-id="@item.Id" class="btn btn-info btn-sm text-white">Details</a>
                <a asp-action="Delete" asp-route-id="@item.Id" class="btn btn-danger btn-sm">Delete</a>
            </td>
        </tr>
    }
</tbody>
</table>
</div>
</div>
</div>
```

Students List [+ Add New Student](#)

Id	Name	Email	Password	Gender	Age	Actions
1	Manahil	manahilabid09@gmail.com	1234567	Female	20	Edit Details Delete
2	Eman	Eman2008@gmail.com	1234	Female	19	Edit Details Delete

• *Create* page:

```
@model DataSubmission.Models.Student
@{
    ViewData["Title"] = "Create Student";
}

<div class="container mt-5">
    <div class="card shadow-lg">
        <div class="card-header bg-success text-white text-center">
            <h3>Create New Student</h3>
        </div>
        <div class="card-body">
            <form asp-action="Create" method="post">
                <div asp-validation-summary="ModelOnly" class="alert alert-danger"></div>
```

```
<div class="mb-3">
    <label asp-for="name" class="form-label fw-bold"></label>
    <input asp-for="name" class="form-control" placeholder="Enter Name" />
    <span asp-validation-for="name" class="text-danger"></span>
</div>
<div class="mb-3">
    <label asp-for="Email" class="form-label fw-bold"></label>
    <input asp-for="Email" class="form-control" placeholder="Enter Email" />
    <span asp-validation-for="Email" class="text-danger"></span>
</div>
<div class="mb-3">
    <label asp-for="Password" class="form-label fw-bold"></label>
    <input asp-for="Password" type="password" class="form-control" placeholder="Enter Password" />
    <span asp-validation-for="Password" class="text-danger"></span>
</div>
</div>
```

```
<label asp-for="Gender" class="form-label fw-bold"></label>
<select asp-for="Gender" class="form-select">
    <option value="">-- Select Gender --</option>
    <option>Male</option>
    <option>Female</option>
</select>
<span asp-validation-for="Gender" class="text-danger"></span>
</div>
<div class="mb-3">
    <label asp-for="age" class="form-label fw-bold"></label>
    <input asp-for="age" type="number" class="form-control" placeholder="Enter Age" />
    <span asp-validation-for="age" class="text-danger"></span>
</div>
<div class="d-flex justify-content-between">
    <a asp-action="Index" class="btn btn-secondary">Back</a>
    <input type="submit" value="Create Student" class="btn btn-success" />
</div>
</div>
```

```
</form>
</div>
</div>
</div>
```

Create New Student

name
Nimra

Email
Eman@gmail.com

Password
....

Gender
-- Select Gender --
-- Select Gender --
Male
Female

[Back](#) [Create Student](#)

- **Details** page:



Student Details

ID:	1
Name:	Manahil
Email:	manahilabid09@gmail.com
Password:	*****
Gender:	Female
Age:	20

Back
Edit

- **Edit** page:



Edit Student

name

Email

Password

Gender

▼

age

Back
Update Student

- **Delete** page:

```
@model DataSubmission.Models.Student
@{
    ViewData["Title"] = "Delete Student";
}

<div class="container mt-5">
    <div class="card shadow-lg border-danger">
        <div class="card-header bg-danger text-white text-center">
            <h3>Delete Student</h3>
        </div>
        <div class="card-body">
            <div class="alert alert-warning text-center">
                <strong>Warning!</strong> Are you sure you want to delete this student?
            </div>
        </div>
    </div>
</div>
```



```
<div class="row mb-3">
    <div class="col-sm-4 fw-bold">Name:</div>
    <div class="col-sm-8">@Model.name</div>
</div>

<div class="row mb-3">
    <div class="col-sm-4 fw-bold">Email:</div>
    <div class="col-sm-8">@Model.Email</div>
</div>

<div class="row mb-3">
    <div class="col-sm-4 fw-bold">Password:</div>
    <div class="col-sm-8">*****</div>
</div>

<div class="row mb-3">
    <div class="col-sm-4 fw-bold">Gender:</div>
    <div class="col-sm-8">@Model.Gender</div>
</div>
```

```
<div class="row mb-4">
    <div class="col-sm-4 fw-bold">Age:</div>
    <div class="col-sm-8">@Model.age</div>
</div>

<div class="text-center">
    <button type="button" class="btn btn-danger px-4" data-bs-toggle="modal" data-bs-target="#confirmDeleteModal">
        Delete
    </button>
    <@asp-action="Index" class="btn btn-secondary px-4">Cancel</@>
</div>
</div>
```



```
<div class="modal fade" id="confirmDeleteModal" tabindex="-1">
    <div class="modal-dialog">
        <div class="modal-content">
            <div class="modal-header bg-danger text-white">
                <div class="modal-title">Confirm Delete</div>
                <button type="button" class="btn-close btn-close-white" data-bs-dismiss="modal"></button>
            </div>
            <div class="modal-body text-center">
                <p>Are you absolutely sure you want to delete this student?</p>
                <div class="text-danger fw-bold">This action cannot be undone.</div>
            </div>
            <div class="modal-footer">
                <@asp-action="Delete" method="post">
                    <button type="button" class="btn btn-danger" data-bs-dismiss="modal">Yes, Delete</button>
                </@>
                <button type="button" class="btn btn-secondary" data-bs-dismiss="modal">No</button>
            </div>
        </div>
    </div>
</div>
```

```
78         </button type="button" class="btn btn-secondary" data-bs-dismiss="modal">No</button>
79     </div>
80 </div>
81 </div>
82 </div>
83 </div>
```

Delete Student

Warning! Are you sure you want to delete this student?

Name: Eman

Email: Eman@gmail.com

Password: *****

Gender: Female

Age: 20

Delete
Cancel

Confirm Delete

Are you absolutely sure you want to delete this student?

This action cannot be undone.

Yes, Delete
No

Name: Eman

Email: Eman@gmail.com

Password: *****

Gender: Female

Age: 20

Delete
Cancel

----- THE END! -----